Trabajo del Módulo de Proyectos

Ciclo Formativo Grado Superior: Administración de sistemas informáticos en red

SERVIJAM

Tutor: Alberto Fernández Sánchez

Autores: Juan Carlos Hernández Risso, Arturo Manso Borrego , Marco Antonio Méndez Rivero

2025

CALIFICACIÓN:	
FECHA:	

D. Alberto Fernández Sánchez, con N.I.F. 50.465.475-W profesor del colegio Institución La Salle, como tutor

C E R T I F I C A (N)

Que la presente memoria titulada:

"Servijam."

Ha sido realizada bajo su dirección por D. Arturo Manso Borrego, con N.I.F. 51.141.572-Z
Ha sido realizada bajo su dirección por D. Juan Carlos Hernández Risso, con N.I.F. 55.063.674-H
Ha sido realizada bajo su dirección por D. Marco Antonio Méndez Rivero, con N.I.F. 51274747-L

Y para que así conste, en cumplimiento de la legislación vigente y a los efectos oportunos firman la presente en Madrid a 10 de junio de 2025.

Agradecimientos

Queríamos agradecer este proyecto a todos los profesores que durante estos dos años han estado a nuestro lado formándonos y ayudándonos a crecer como profesionales en este sector.

Licencia

© Esta obra está bajo una licencia de Creative Commons Reconocimiento-NoComercial 4.0 Internacional License

Resumen

El presente proyecto, titulado **SERVIJAM**, consiste en el diseño e implementación de un sistema de gestión para impresoras 3D en red, orientado a entornos colaborativos como centros educativos, laboratorios de prototipado o espacios maker. La solución se basa en una **arquitectura modular**, construida sobre una Raspberry Pi y apoyada en tecnologías como **Klipper**, **Moonraker**, **Fluidd**, **Docker** y **Tailscale**, que permiten el control remoto, seguro y eficiente de los procesos de impresión.

El sistema incluye una **interfaz web** desarrollada desde cero que facilita el **registro de usuarios**, la **carga** y **selección de archivos G-code**, el seguimiento del estado de la impresora en tiempo real, así como la gestión de la cola de impresión y el envío de notificaciones automáticas. Se ha puesto especial **énfasis** en la **seguridad del sistema**, la **trazabilidad** de los trabajos y la **integración** transparente en redes ya existentes, sin comprometer la estabilidad de otros servicios.

A nivel experimental, se han desarrollado **paneles diferenciados** para **usuarios** y **administradores**, permitiendo un control granular sobre las operaciones. La validación se realizó en un **entorno simulado de producción**, confirmando la **eficacia**, **escalabilidad** y **facilidad de uso** del sistema. **SERVIJAM** se consolida así como una **plataforma completa**, **adaptable** y **replicable** para la gestión profesional de servicios de impresión 3D en red.

Palabras clave

Impresión 3D, gestión remota, Raspberry Pi, Klipper, Moonraker, Fluidd, Tailscale, automatización, red local, servidor web, G-code, interfaz web, seguridad informática, administración de sistemas, proyectos integradores, Docker, Nginx Proxy Manager, Duck DNS, desarrollo full stack, control de impresión, virtualización de servicios, cola de trabajos, panel de administración, conectividad segura, acceso autenticado, monitorización en tiempo real, código abierto, eficiencia operativa, entorno colaborativo, infraestructura de red, trazabilidad digital.

Índice General

Capítulo 1. Introducción

1.1. Objetivo del proyecto 7
1.2. Requisitos del proyecto..... 7

Capítulo 2. Base teórica

2.1. Esquema de red..... 9
2.2. Creación del entorno de impresión 3D..... 9
2.3. Aplicación a una red ya existente..... 18

Capítulo 3. Descripción experimental

3.1. Creación de web interactiva..... 19
3.2. Base de datos..... 54

Capítulo 4. Conclusiones y líneas futuras

4.1. CONCLUSIONES..... 58
4.2. LÍNEAS FUTURAS..... 59

Bibliografía

Capítulo 1
INTRODUCCIÓN

1.1. Objetivo del proyecto

El presente proyecto tiene como objetivo principal el diseño, la implementación y el despliegue de un servidor de impresión 3D en una red interna, orientado a optimizar el flujo de trabajo en entornos colaborativos que requieran acceso controlado y eficiente a una o varias impresoras 3D.
La solución propuesta busca centralizar, automatizar y asegurar la gestión de trabajos de impresión mediante una interfaz web intuitiva, proporcionando una plataforma eficaz, escalable y adaptable a contextos como instituciones educativas, laboratorios de prototipado o espacios maker de uso interno.

1.2. Requisitos del proyecto

A continuación, se describen los requisitos funcionales y técnicos que han guiado el desarrollo del sistema propuesto:

Requisitos Funcionales

Registro y autenticación de usuarios
El sistema permitirá el registro de nuevos usuarios, así como la autenticación mediante credenciales. La protección de los datos personales se garantizará mediante el uso de técnicas de cifrado, asegurando su integridad y confidencialidad.

Carga de archivos .gcode
Los usuarios autenticados podrán subir archivos con extensión .gcode a través de una interfaz web desarrollada con tecnologías estándar como HTML, CSS y JavaScript. El sistema validará tanto el tipo como el tamaño de los archivos antes de aceptar su almacenamiento.

Gestión de cola de impresión
La plataforma incorporará una cola de impresión gestionada bajo la política FIFO (First In, First Out), lo que permitirá organizar automáticamente los trabajos en función del momento en que fueron enviados.

Restricción de acceso a la plataforma
El acceso al sistema estará limitado exclusivamente a los usuarios autorizados de la organización. Esta restricción se aplicará mediante mecanismos como listas blancas de direcciones IP, validación de credenciales o integración con un servicio de directorio interno.

Monitorización en tiempo real
El sistema integrará funciones de monitorización en tiempo real del estado de la impresora 3D, permitiendo visualizar parámetros clave como la temperatura del hotend y la cama, el progreso del trabajo y posibles errores. Esta información se obtendrá mediante peticiones a la API REST de Moonraker.

Panel de administración

Se dispondrá de un panel de control exclusivo para administradores, desde el cual se podrá:

- Visualizar en detalle el estado de la impresora.
- Gestionar la cola de impresión (pausar, reordenar o cancelar trabajos).
- Modificar parámetros de impresión en tiempo real, tales como la velocidad, la temperatura o la aceleración.

Notificaciones automáticas

El sistema contará con un servicio de notificaciones por correo electrónico que informará a los usuarios sobre el estado de sus trabajos de impresión (inicio, finalización o errores), con el fin de mantenerlos informados en todo momento.

Requisitos Técnicos

Infraestructura de hardware

El sistema se implementará sobre una Raspberry Pi 4 (o superior), conectada directamente a la impresora 3D mediante un puerto USB. La impresora deberá contar con el firmware KLIPPER correctamente configurado para garantizar la compatibilidad con el software del sistema.

Sistema operativo y software base

La Raspberry Pi utilizará el sistema operativo Raspbian, y se instalarán los siguientes componentes:

- Klipper: Firmware para el control de la impresora.
- Moonraker: Servidor intermedio encargado de exponer una API REST.
- Fluidd: Interfaz gráfica de usuario para la visualización del estado de la impresora.

Base de datos local

La gestión de usuarios, archivos y trabajos de impresión se realizará mediante una base de datos relacional local. Dependiendo de las necesidades de escalabilidad y concurrencia, se optará por SQLite o MariaDB como sistema de gestión de base de datos.

Servidor web

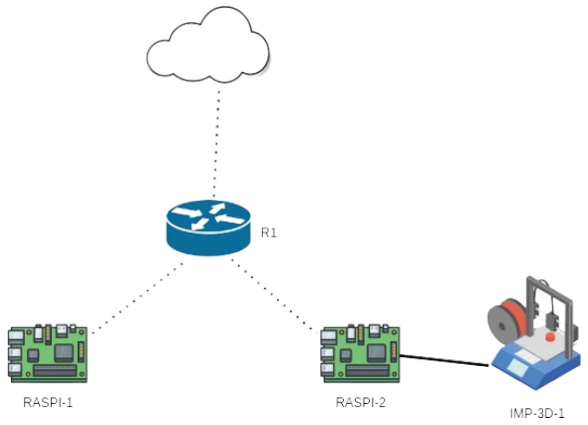
La interfaz web será servida mediante un servidor Apache instalado y configurado localmente, garantizando su disponibilidad dentro de la red interna de la organización.

Gestión de archivos

Los archivos .gcode serán almacenados en una estructura jerárquica organizada por usuario. Esta organización permitirá garantizar la trazabilidad, la integridad de los datos y el cumplimiento de las políticas de acceso y seguridad definidas.

Capítulo 2
BASE TEÓRICA

2.1. Esquema de red



R1: Router doméstico
RASPI-1: Gestor de servicios internos y externos
RASPI-2: Controlador de impresión
IMP-3D-1: Impresora FDM

2.2. Creación del entorno de impresión 3D

Para la implementación del entorno de impresión 3D en red, ha sido necesario configurar una infraestructura que garantice la conectividad segura, eficiente y privada entre los distintos dispositivos implicados.
Uno de los componentes clave en esta arquitectura es el servicio de red privada virtual (VPN) ofrecido por **Tailscale**, que permite una interconexión directa entre nodos sin necesidad de configuraciones complejas a nivel de red.

TAILSCALE

Tailscale es una solución de red privada virtual de tipo *overlay* basada en el protocolo **WireGuard**, diseñada para simplificar la creación de redes punto a punto entre dispositivos distribuidos geográficamente. Su arquitectura elimina la necesidad de abrir puertos o configurar reglas de reenvío en routers, y se apoya en mecanismos de autenticación modernos como OAuth o Single Sign-On (SSO), así como en la criptografía de clave pública.

Funcionamiento de Tailscale

El funcionamiento de Tailscale se puede dividir en dos fases principales: el establecimiento de túneles seguros mediante WireGuard y la travesía de NAT (NAT Traversal) para permitir conexiones directas entre dispositivos.

1. Establecimiento de Túneles WireGuard

La comunicación segura entre dispositivos se establece a través del protocolo WireGuard, siguiendo las siguientes etapas:

Generación de claves criptográficas

Cada dispositivo que desea unirse a la red privada debe generar un par de claves: una clave pública y una clave privada. Este par es esencial para el cifrado de las comunicaciones y la autenticación entre nodos.

Comunicación con el servidor de coordinación de Tailscale

Una vez generadas las claves, el dispositivo se comunica con el servidor de coordinación (control plane) de Tailscale, enviando su clave pública. Este servidor no participa directamente en el flujo de datos, pero sí en la gestión inicial de la red.

Asignación de dirección IP virtual

El servidor de coordinación asigna al dispositivo una dirección IP virtual interna dentro del rango gestionado por Tailscale (por ejemplo, 100.74.3.6). Esta IP será utilizada para identificar al nodo dentro de la red privada. La clave pública del dispositivo queda asociada a dicha dirección, facilitando las futuras conexiones con otros nodos.

Establecimiento de conexión entre nodos

Cuando otro dispositivo desea comunicarse con el nodo registrado, el servidor de coordinación le proporciona la dirección IP virtual y la clave pública necesarias. Con esta información, ambos nodos pueden establecer un túnel cifrado de manera directa utilizando WireGuard, garantizando así la privacidad y seguridad de las comunicaciones.

2. NAT Traversal y Conexión Directa entre Nodos

Una de las principales dificultades en la conexión entre dispositivos distribuidos en distintas redes es la presencia de mecanismos de traducción de direcciones de red (NAT). Tailscale emplea técnicas de NAT Traversal para facilitar la conexión directa entre nodos, incluso cuando ambos se encuentran detrás de routers NAT.

Envío de paquetes de señalización

Ambos dispositivos envían paquetes UDP iniciales al servidor de coordinación. Estos paquetes no contienen datos útiles, sino que sirven para informar al servidor sobre la presencia y configuración de red de cada nodo.

Identificación de IP pública y puerto de origen

El servidor analiza estos paquetes y registra la dirección IP pública y el puerto asignado por el router NAT a cada nodo. Esta información es vital para que los dispositivos puedan encontrarse mutuamente a través de Internet.

Intercambio de información entre pares

Una vez obtenida la información de red de ambos dispositivos, el servidor de coordinación facilita el intercambio, informando a cada nodo sobre la dirección (IP:PUERTO) del otro. Esto permite que ambos inicien una conexión directa entre sí.

Establecimiento de la conexión punto a punto

Con la información proporcionada, los nodos intentan establecer una comunicación directa, atravesando el NAT de forma segura y automática. Una vez que la conexión se establece correctamente, el tráfico entre dispositivos deja de depender del servidor de coordinación, lo que mejora el rendimiento, reduce la latencia y mantiene la privacidad de extremo a extremo.

MOONRAKER

Moonraker es una API desarrollada en lenguaje Python cuya principal función es actuar como intermediario entre el firmware Klipper y las interfaces web de gestión de impresoras 3D, como Fluidd o Mainsail. Este componente se ejecuta en la misma máquina que Klipper (en este caso, en la Raspi-2) y permite la interacción remota con la impresora mediante solicitudes HTTP, ofreciendo así un entorno de control accesible, extensible y adaptado a entornos conectados.

Funcionamiento de Moonraker

Moonraker proporciona una capa de abstracción que facilita la consulta del estado de la impresora, el control de las tareas de impresión y la recepción de eventos en tiempo real, todo ello a través de un conjunto de interfaces bien definidas. Sus principales funcionalidades se pueden dividir en tres apartados:

1. Consulta del Estado de la Impresora

A través de su API, Moonraker ofrece acceso a información detallada y actualizada sobre el estado operativo de la impresora 3D. Entre los datos que proporciona se encuentran la **temperatura del extrusor** y de la **cama caliente**, la **posición de los ejes**, el estado de los sensores conectados y el **progreso** exacto de la **impresión en curso**. Esta capacidad resulta fundamental para una supervisión remota eficaz, ya que permite detectar posibles anomalías, verificar que los parámetros de funcionamiento sean correctos y, en caso necesario, realizar ajustes para garantizar la calidad y seguridad del proceso de impresión.

2. Control de Tareas de Impresión

La API de Moonraker no solo permite consultar el estado de la impresora, sino que también ofrece control directo sobre las tareas de impresión. A través de ella, es posible iniciar un trabajo, así como pausar, reanudar o cancelar en cualquier momento. Además, facilita el seguimiento en tiempo real del avance de la impresión, lo que permite mantener un control preciso del proceso sin necesidad de intervención física. Esta funcionalidad resulta especialmente valiosa en entornos donde la impresora se encuentra en una ubicación diferente al usuario, ya que posibilita una gestión remota eficiente, centralizada y segura.

3. Actualizaciones en Tiempo Real mediante WebSocket

Además de las solicitudes convencionales mediante HTTP, Moonraker incorpora soporte para conexiones WebSocket, lo que permite establecer un canal de comunicación bidireccional y persistente con la impresora. Gracias a esta funcionalidad, el sistema puede enviar de forma automática notificaciones sobre eventos relevantes y actualizar en tiempo real información clave como las temperaturas, el estado actual del proceso de impresión, posibles errores y la finalización de las tareas. Esta capacidad elimina la necesidad de recargar la interfaz web constantemente y garantiza una experiencia de usuario ágil, dinámica y totalmente adaptada a los requerimientos de los entornos de impresión 3D modernos y conectados.

FLUIDD

Fluidd es una interfaz web ligera y de código abierto diseñada para gestionar impresoras 3D que utilizan el firmware Klipper en conjunto con Moonraker. Su principal función es ofrecer una experiencia visual y accesible desde cualquier navegador, permitiendo controlar la impresora de forma remota a través de una red local o una VPN como Tailscale. Al no requerir instalaciones complejas ni software adicional, Fluidd se presenta como una solución óptima para entornos domésticos y profesionales donde se prioriza la simplicidad, el rendimiento y la accesibilidad multiplataforma.

Funcionamiento de Fluidd

A pesar de ser la interfaz principal de usuario, Fluidd no se comunica directamente con Klipper. Su funcionamiento se basa en una arquitectura en capas, donde la interacción con la impresora se realiza a través de Moonraker, que actúa como intermediario entre la interfaz web y el firmware. Este enfoque modular permite separar la lógica de presentación de la lógica de control, facilitando así el mantenimiento y la escalabilidad del sistema.

1.Interacción con Moonraker

Cuando el usuario accede a Fluidd desde su navegador, ya sea mediante la dirección http://fluidd.local o a través de la IP local de la Raspberry Pi, se establece una conexión con el servidor Moonraker. Esta comunicación se realiza mediante solicitudes HTTP y conexiones WebSocket, lo que permite tanto el envío de comandos como la recepción de datos en tiempo real. Gracias a este sistema, Fluidd puede mostrar información actualizada al instante sin necesidad de refrescar la página.

2.Obtención y visualización de datos

Moonraker, una vez recibe las peticiones desde Fluidd, se encarga de consultar el estado de la impresora a través de Klipper. Esta consulta abarca desde archivos y sensores, hasta parámetros del sistema operativo. A continuación, los datos recopilados se devuelven a Fluidd, que los procesa y presenta al usuario en un entorno gráfico intuitivo. Esta información incluye temperaturas, estado de los ejes, progreso de impresión, logs, alertas y mucho más.

3.Gestión completa del proceso de impresión

Desde Fluidd, el usuario puede iniciar, pausar, cancelar y monitorear impresiones de manera remota. También es posible cargar archivos .gcode, modificar configuraciones del sistema, controlar ventiladores o luces conectadas, y acceder a gráficas de seguimiento en tiempo real. Todo esto se integra en una interfaz dinámica y responsiva, compatible con dispositivos móviles, tablets y ordenadores de escritorio.

En conjunto, Fluidd representa la capa visual y de interacción del ecosistema Klipper-Moonraker, ofreciendo al usuario una herramienta potente y fácil de usar para el control completo de sus impresiones 3D, con una experiencia optimizada para redes locales y entornos conectados.

APACHE HTTP SERVER

Apache HTTP Server, comúnmente conocido como **Apache**, es un servidor web de código abierto ampliamente utilizado en entornos de desarrollo y producción. Su objetivo principal es permitir la entrega de contenido web a través del protocolo HTTP (y HTTPS), facilitando la publicación y administración de sitios web en redes locales. Gracias a su arquitectura modular, su compatibilidad con múltiples sistemas operativos y su robustez, Apache se posiciona como una solución versátil tanto para pequeñas implementaciones como para infraestructuras empresariales complejas.

Funcionamiento de Apache

Apache actúa como el intermediario entre los clientes que realizan solicitudes web (por ejemplo, navegadores) y el contenido alojado en un servidor. Su funcionamiento se basa en un sistema modular, en el que se cargan y configuran extensiones específicas según las necesidades del entorno. Esta arquitectura permite una alta personalización, manteniendo al mismo tiempo la eficiencia y la estabilidad del servicio.

1. Interacción con el cliente

Cuando un usuario accede a una dirección web gestionada por Apache, por ejemplo mediante http://www.ejemplo.local o la IP del servidor, el navegador envía una solicitud HTTP al puerto configurado (generalmente el puerto 80 para HTTP o 443 para HTTPS). Apache recibe esta solicitud y, mediante su motor de procesamiento y los módulos activos, determina cómo debe responder. Esta respuesta puede consistir en la entrega de archivos estáticos (HTML, imágenes, hojas de estilo), la ejecución de scripts dinámicos (como PHP, mediante módulos como mod_php o mod_proxy_fcgi), o la redirección hacia otros servicios internos o externos.

2. Obtención y entrega de contenido

Una vez procesada la solicitud, Apache accede al sistema de archivos del servidor para localizar los recursos requeridos, ya sea una página web, un archivo multimedia o el resultado de una aplicación web. Estos recursos son entregados al cliente siguiendo los estándares del protocolo HTTP, incluyendo cabeceras que definen el tipo de contenido, el estado de la respuesta y otros metadatos. Además, si el servidor emplea certificados SSL/TLS, Apache puede cifrar la comunicación para garantizar la confidencialidad e integridad de los datos transmitidos.

3. Gestión completa del servicio web

Apache permite una administración detallada del servicio mediante archivos de configuración como httpd.conf o archivos .htaccess, que controlan aspectos como redirecciones, autenticación de usuarios, compresión de contenido, control de acceso por IP, entre otros. Desde el punto de vista del administrador, también es posible supervisar el rendimiento del servidor mediante logs detallados, ajustar la carga de trabajo a través de distintos MPMs (Multi-Processing Modules), y realizar cambios en caliente sin necesidad de reiniciar completamente el sistema. Su compatibilidad con herramientas externas y paneles de gestión, así como con sistemas de automatización y contenedores, amplía aún más sus capacidades.

DOCKER

Docker es una plataforma de código abierto que ha revolucionado la forma en que los desarrolladores y equipos de operaciones construyen, distribuyen y ejecutan aplicaciones. Su función principal es la **contenerización**, un proceso que permite empaquetar una aplicación y todas sus dependencias (bibliotecas, configuraciones, etc.) en unidades aisladas llamadas **contenedores**. Estos contenedores son ligeros, portables y consistentes, garantizando que una aplicación funcione de la misma manera en cualquier entorno, desde el desarrollo local hasta la producción en la nube.

Funcionamiento de Docker

El funcionamiento de Docker se basa en el concepto de **imágenes** y **contenedores**. Una imagen Docker es una plantilla inmutable que contiene el código de la aplicación, un sistema operativo base, las bibliotecas y las dependencias necesarias. Los contenedores son instancias ejecutables de estas imágenes, proporcionando un entorno aislado y autocontenido para la aplicación.

1.Imágenes Docker

La Base de la Contenerización Una imagen Docker se construye a partir de un **Dockerfile**, un archivo de texto que contiene una serie de instrucciones para crear la imagen. Estas instrucciones definen el sistema operativo base, los paquetes a instalar, el código de la aplicación, las configuraciones de red, las variables de entorno y los comandos que se ejecutarán al iniciar el contenedor. Una vez creada, la imagen se puede almacenar en un registro público (como Docker Hub) o privado, facilitando su distribución y reutilización. La inmutabilidad de las imágenes garantiza que el entorno de ejecución de la aplicación sea idéntico en todas partes.

2.Contenedores

Instancias Aisladas y Portables Un contenedor Docker es una instancia ligera, aislada y ejecutable de una imagen. Cada contenedor funciona como un proceso independiente en el sistema operativo del host, pero con su propio sistema de archivos, procesos y recursos de red, lo que evita conflictos entre aplicaciones y garantiza la consistencia del entorno. El aislamiento a nivel del kernel del sistema operativo proporciona una eficiencia superior a la de las máquinas virtuales tradicionales, ya que los contenedores comparten el mismo kernel del host en lugar de virtualizar uno completo.

3.El Motor Docker (Docker Engine)

Orquestación del Ciclo de Vida El corazón de Docker es el **Docker Engine**, un demonio que se ejecuta en el sistema operativo del host y es responsable de construir imágenes, crear y ejecutar contenedores, gestionar redes y volúmenes, y supervisar el ciclo de vida de los contenedores. Los usuarios interactúan con el Docker Engine a través de la interfaz de línea de comandos (CLI) de Docker o mediante la API de Docker.

DOCKER-COMPOSE

Docker Compose es una herramienta complementaria a Docker que facilita la definición y ejecución de aplicaciones multicontenedor. Permite orquestar múltiples servicios que trabajan juntos para formar una aplicación completa, como un servidor web, una base de datos y un balanceador de carga. Mediante un único archivo de configuración YAML, Docker Compose permite definir la arquitectura de la aplicación, las dependencias entre los servicios, las redes y los volúmenes, simplificando enormemente el despliegue y la gestión de entornos complejos.

Funcionamiento de Docker Compose

Docker Compose simplifica la gestión de aplicaciones compuestas por múltiples servicios contenerizados. Su funcionamiento se centra en un archivo YAML, convencionalmente llamado docker-compose.yml, que describe la arquitectura completa de la aplicación.

1.Definición de Servicios

El archivo docker-compose.yml define los servicios que componen la aplicación, donde cada servicio representa un contenedor. Para cada servicio, se especifica la imagen Docker a utilizar, los puertos a mapear, los volúmenes a montar, las variables de entorno, las dependencias de otros servicios y cualquier configuración específica del contenedor. Esta definición centralizada elimina la necesidad de ejecutar múltiples comandos docker run con parámetros complejos, agilizando el despliegue y la reproducción de entornos.

2.Orquestación de la Aplicación Multicontenedor

Una vez definido el archivo docker-compose.yml, un único comando (docker-compose up) es suficiente para construir las imágenes (si es necesario), crear las redes, levantar los contenedores en el orden correcto según sus dependencias, y configurar los volúmenes. Docker Compose se encarga de la orquestación, asegurando que los servicios se inicien en la secuencia adecuada y puedan comunicarse entre sí a través de la red interna de Docker.

3.Gestión Centralizada del Ciclo de Vida

Docker Compose no solo facilita el despliegue inicial, sino también la gestión del ciclo de vida de la aplicación. Permite detener (docker-compose down), reiniciar, escalar y ver el estado de todos los servicios de la aplicación de manera centralizada. Esto es crucial en entornos de desarrollo donde se requiere reproducir rápidamente configuraciones complejas, y en producción, donde la gestión coherente de la aplicación multicontenedor es fundamental.

DUCK DNS

Duck DNS es un servicio de DNS dinámico (DDNS) gratuito que permite a los usuarios asociar un nombre de subdominio personalizado bajo **duckdns.org** con una dirección IP pública que cambia con el tiempo. En entornos domésticos o de pequeñas oficinas, los Proveedores de Servicios de Internet (ISP) suelen asignar direcciones IP dinámicas, lo que dificulta el acceso remoto a servidores o dispositivos alojados en esas redes, ya que la IP puede cambiar en cualquier momento. Duck DNS resuelve este problema al proporcionar una forma de mantener actualizado el registro DNS de un subdominio con la IP actual del usuario.

Funcionamiento de Duck DNS

El funcionamiento de Duck DNS se basa en la constante actualización del registro DNS de un subdominio con la dirección IP pública actual del usuario. Este proceso se realiza de manera automatizada para asegurar que el nombre de dominio siempre apunte a la IP correcta.

1.Registro y Subdominio

El primer paso es registrarse en el sitio web de Duck DNS y crear uno o varios subdominios (hasta cinco de forma gratuita) bajo duckdns.org. Por ejemplo, micasa.duckdns.org. Al registrar un subdominio, Duck DNS genera un "token" único para la cuenta, el cual es esencial para autenticar las solicitudes de actualización de IP.

2.Detección de la IP Pública

Para que Duck DNS funcione, un dispositivo en la red del usuario (puede ser un router compatible, un servidor, una Raspberry Pi, o incluso un contenedor Docker) necesita determinar su dirección IP pública actual. Esta IP es la que el ISP asigna al router y es la visible desde internet. Los métodos para obtener esta IP varían, pero comúnmente implican realizar una consulta a un servicio web externo diseñado para devolver la IP pública del cliente.

3.Mecanismo de Actualización

Una vez que la dirección IP pública ha sido obtenida, el dispositivo envía una solicitud HTTP (generalmente un GET) a la API de Duck DNS. Esta solicitud incluye el subdominio a actualizar y el token de autenticación. Duck DNS recibe esta solicitud y, si la autenticación es exitosa, actualiza el registro DNS de ese subdominio en sus servidores para que apunte a la nueva dirección IP.

4.Resolución de Nombres de Dominio

Una vez que Duckdns ha actualizado el registro DNS, cuando un usuario intenta acceder a tudominio.duckdns.org desde cualquier parte de internet, el sistema de resolución de nombres de dominio (DNS) lo dirigirá a la dirección IP pública que Duckdns tiene registrada en ese momento. Esto permite que el tráfico llegue al router del usuario y, si se han configurado correctamente las redirecciones de puertos (port forwarding) en el router, al servicio o dispositivo deseado dentro de la red local.

NGINX PROXY MANAGER

Nginx Proxy Manager (NPM) es una herramienta de código abierto que simplifica enormemente la gestión de proxys inversos basados en NGINX a través de una interfaz de usuario web intuitiva. Su objetivo principal es proporcionar una manera fácil de exponer servicios web que se ejecutan en su red local (o en contenedores) a Internet, gestionar certificados SSL/TLS (incluyendo Let's Encrypt para HTTPS gratuito), y redirigir el tráfico a los destinos correctos.

Funcionamiento de Nginx Proxy Manager

Nginx Proxy Manager actúa como un intermediario inteligente entre los clientes que acceden a tus servicios web (desde Internet) y los servidores reales donde esos servicios están alojados (en tu red local o en otros contenedores). Su funcionamiento se basa en tres pilares: **Nginx como motor de proxy**, una **base de datos** para almacenar las configuraciones y una **interfaz web** para la gestión.

1.Motor NGINX (Interno)

En su núcleo, NPM utiliza una instancia de NGINX. Cuando configuras un "Host Proxy" a través de la interfaz web de NPM, lo que realmente estás haciendo es generar automáticamente la configuración de NGINX necesaria en segundo plano. NGINX es un servidor web y proxy inverso de alto rendimiento, conocido por su eficiencia, estabilidad y capacidad para manejar un gran número de conexiones concurrentes. NPM aprovecha estas capacidades para dirigir el tráfico de manera eficiente.

2.Base de Datos para Almacenar Configuraciones

Toda la configuración que realizamos en la interfaz web de NPM (dominios, destinos, certificados SSL, etc.) Se almacena en una base de datos interna (generalmente SQLite por defecto, o MySQL/MariaDB para configuraciones más robustas). Esta base de datos es esencial porque NPM lee de ella para generar y aplicar las configuraciones de NGINX. Cuando inicias o reinicias NPM, o cuando aplicas cambios, la lógica de NPM consulta la base de datos para construir los archivos de configuración de NGINX de forma dinámica.

2.3. Aplicación a una red ya existente

La solución propuesta ha sido diseñada para integrarse de forma transparente en redes locales previamente desplegadas, sin necesidad de realizar cambios disruptivos en la infraestructura existente. Para ello, se han seguido una serie de principios de compatibilidad, escalabilidad y mínima intervención, lo que permite implementar el sistema de impresión 3D en entornos donde ya conviven otros servicios, usuarios y dispositivos.

Integración con servicios de red existentes

El servidor de impresión 3D, implementado sobre una Raspberry Pi, ha sido configurado con una dirección IP estática dentro del rango de la red local existente, lo que garantiza su disponibilidad constante y facilita su localización por parte de los dispositivos conectados. Adicionalmente, se ha habilitado la herramienta Tailscale como solución de conectividad remota, permitiendo integrar el servidor en otras redes de forma segura y sin necesidad de modificar la configuración de red local.

Mantenimiento del esquema de seguridad

La incorporación del servidor no compromete la seguridad de la red, ya que opera bajo políticas de restricción de acceso preexistentes. A través del uso de listas de control de acceso (ACLs) en el router o firewall local, se limita el acceso a la interfaz web a usuarios autorizados. La autenticación en la plataforma se apoya en credenciales individuales cifradas, y el tráfico puede cifrarse mediante HTTPS si se instala un certificado TLS.

Convivencia con servicios ya desplegados

El servidor web Apache ha sido configurado para evitar conflictos con otros servicios que pudieran estar utilizando el mismo puerto. En entornos donde ya existe un servidor web corporativo, se puede utilizar un puerto alternativo (como el 8080) o bien definir un VirtualHost específico dentro del archivo de configuración de Apache. Esto permite alojar la plataforma de impresión como un servicio adicional sin interferir con las aplicaciones existentes.

Adaptabilidad a redes de bajo rendimiento

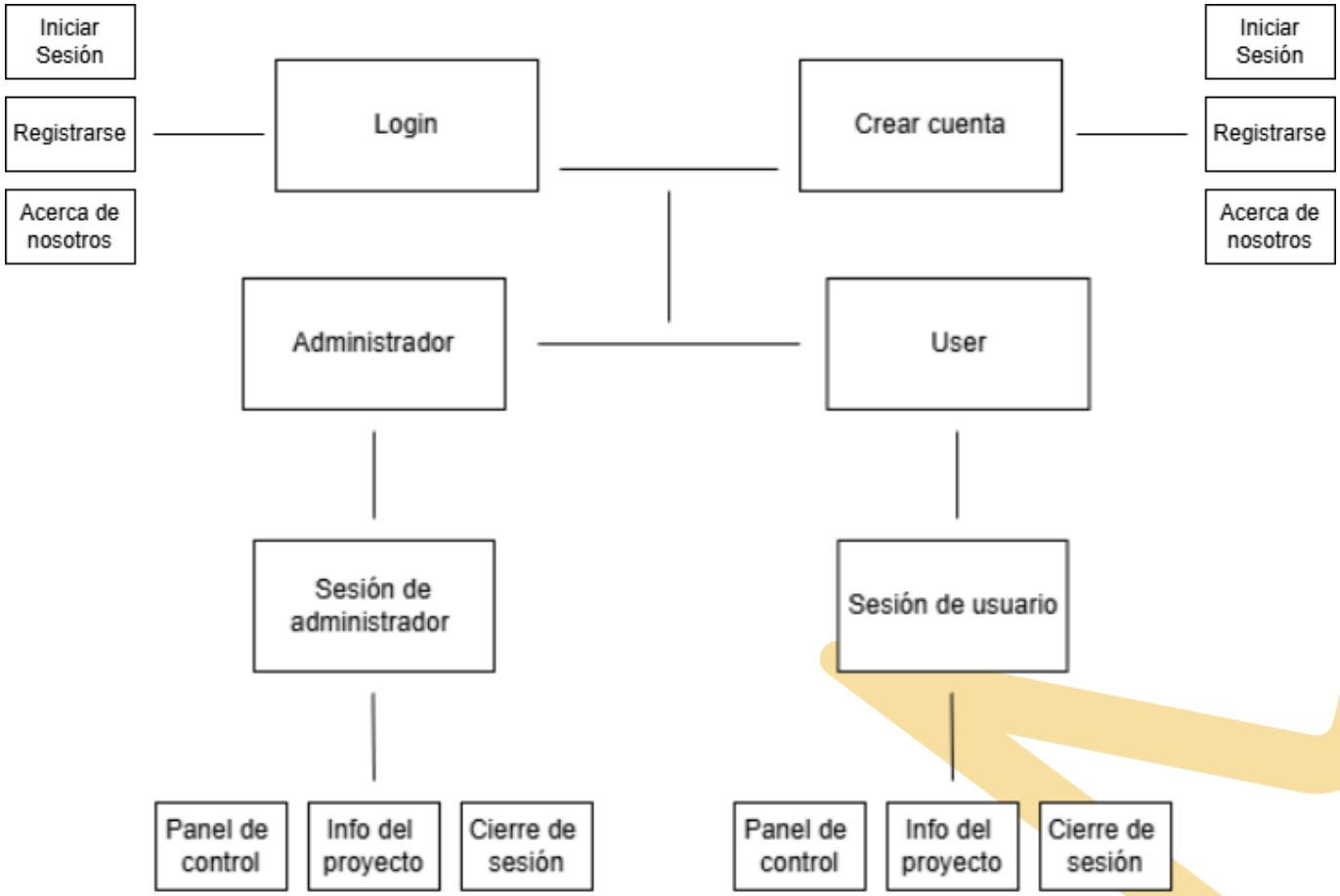
Considerando que muchas redes preexistentes pueden tener limitaciones de ancho de banda o estabilidad, se ha optimizado el sistema para reducir la carga de red. La interfaz Fluidd es ligera y eficiente, mientras que Moonraker permite actualizaciones selectivas mediante WebSocket sin necesidad de recargar páginas ni realizar solicitudes innecesarias.

Capítulo 3
DESCRIPCIÓN EXPERIMENTAL

3.1. Creación de web interactiva

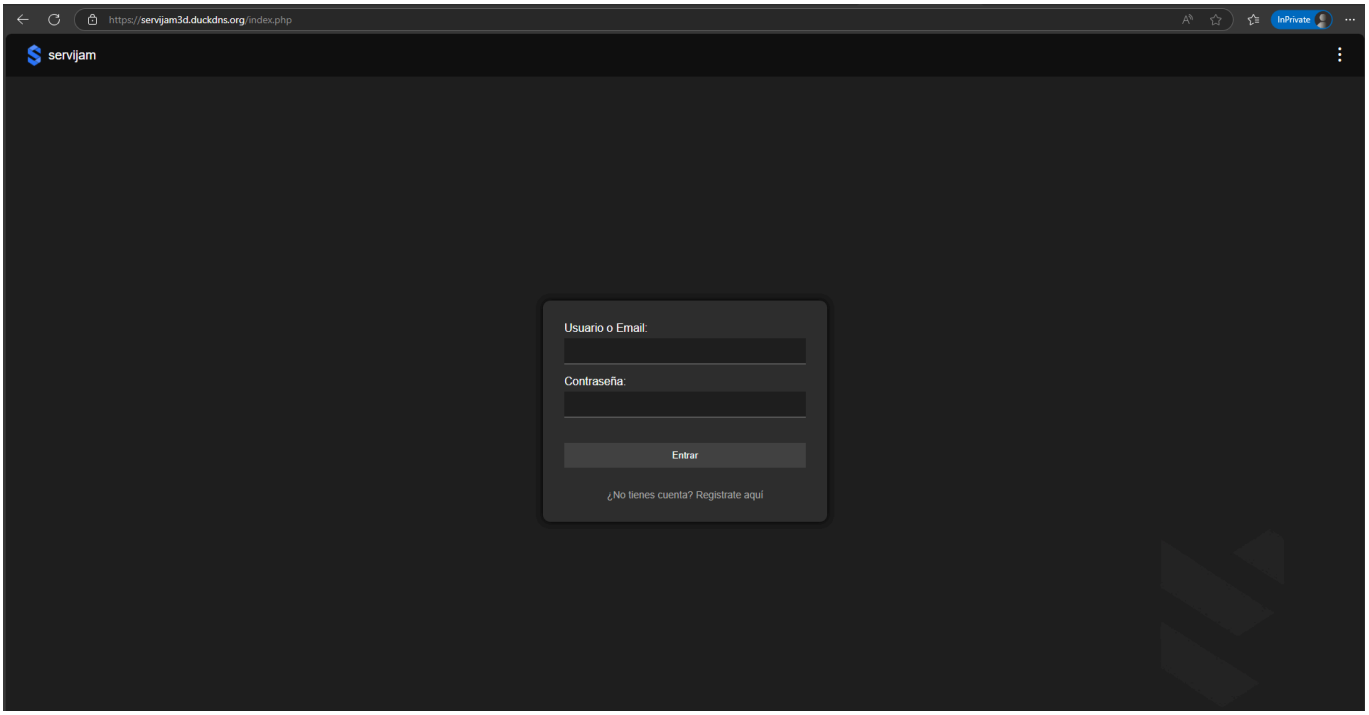
El proyecto "Servijam" es una aplicación web diseñada para la gestión de trabajos de impresión 3D. Su objetivo principal es simplificar el envío de archivos Gcode para impresión, ofreciendo un control centralizado para administradores y usuarios finales. La plataforma busca automatizar y optimizar el flujo de impresión, mejorando la organización, el control de acceso y la trazabilidad de los trabajos.

Estructura

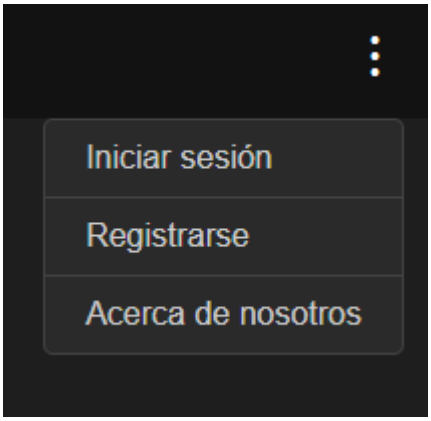


Inicio de sesión.

En la interfaz de inicio de sesión contamos con un recuadro en el cual el usuario deberá de introducir sus credenciales **(nombre de usuario/correo electrónico y contraseña)** para validar la sesión. Una vez introducido los datos si el usuario y contraseña existe en la base de datos el usuario accede a su sesión.

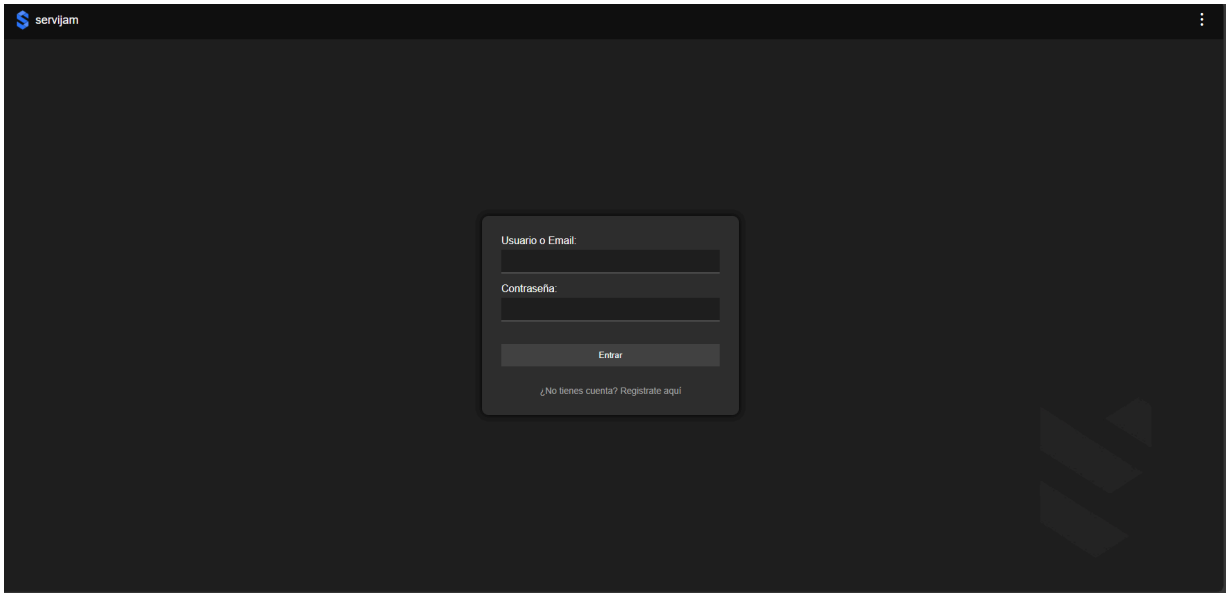


En la parte superior a la derecha contamos con tres puntos, si pinchamos sobre se abrirá un pequeño menú con varias opciones.

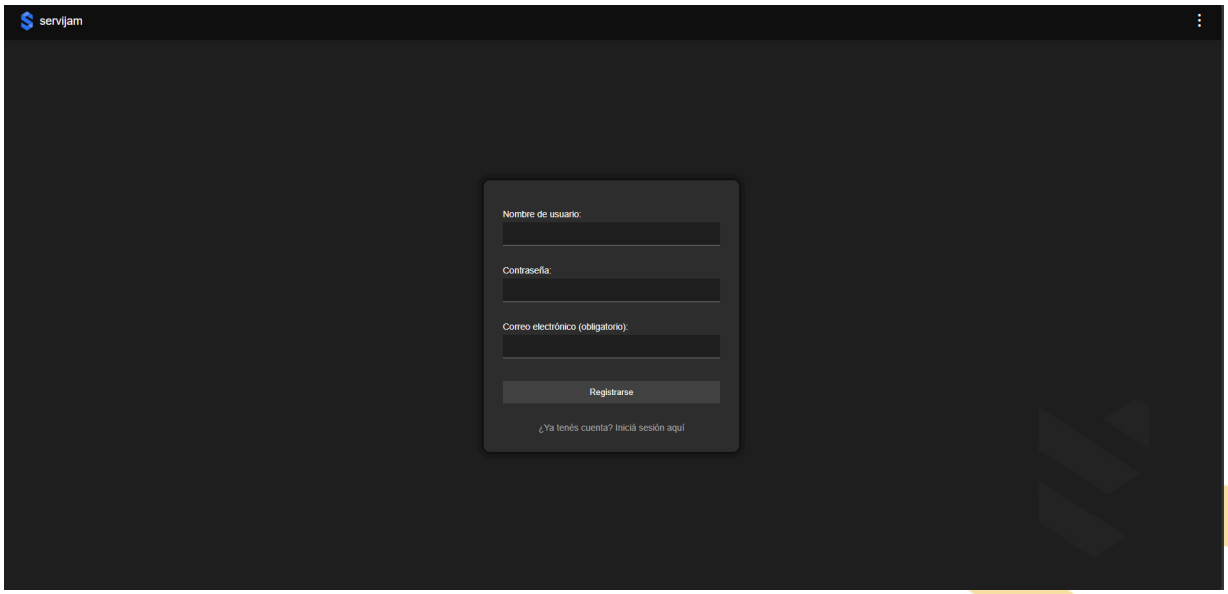


Este menú nos ofrece las siguientes opciones:

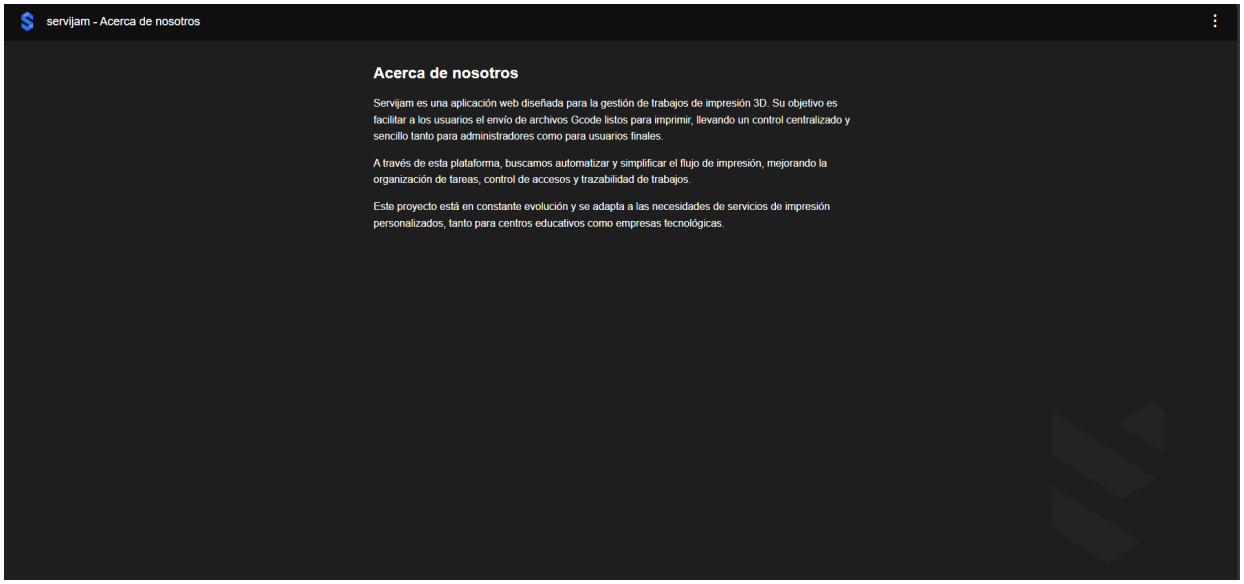
Iniciar sesión. Nos lleva al inicio de sesión.



Registrarse. Nos lleva al formulario de registro.



Acerca de nosotros. Nos lleva a un apartado donde contamos nuestro propósito con **SERVIJAM**.



<u>Código de la página Iniciar Sesión</u>

Este trozo de código cuenta con la barra superior, el menú, el recuadro de nombre de usuario o correo electrónico, el recuadro de la contraseña y el enlace a la página de registro.

```
<body>

    <div class="topbar">
        <div class="logo">
            <img src="imgs/logo.jpg" alt="Logo">
        </div>
        <div class="title">servijam</div>
        <div class="menu-button" onclick="toggleMenu()">⋮</div>
        <div class="dropdown" id="dropdown-menu">
            <div onclick="location.href='index.php'">Iniciar sesión</div>
            <div onclick="location.href='register.php'">Registrarse</div>
            <div onclick="location.href='about.php'">Acerca de nosotros</div>
        </div>

    </div>


    <div class="login-box">
        <form class="login-form" action="login.php" method="post">
            <label>Usuario o Email:</label>
            <input type="text" name="user_input" required>
            <label>Contraseña:</label>
            <input type="password" name="password" required>
            <input type="submit" value="Entrar">
            <a href="register.php">¿No tienes cuenta? Registrate aquí</a>
        </form>
    </div>
```

Función para iniciar la sesión.

Este bloque gestiona el inicio de sesión según los roles del usuario **(administrador/cliente)**. Trabaja con inyecciones SQL, validación de datos, hash de contraseñas. Primero, conecta con la base de datos y comprueba la existencia y coincidencia de las credenciales almacenadas en ella.

Si la contraseña en la base de datos no está encriptada genera un hash para ella y actualiza la BD.

Una vez verificadas las credenciales, guarda los datos del usuario **(nombre, rol, ID)** en la sesión y redirige al usuario según el rol que tenga asignado en dicho campo **(user/admin)**, a admin.php si es administrador y user.php en casos de clientes.

Si no pasa la verificación, el usuario es redirigido con un mensaje de error.

```
session_start();
require 'db.php';

$user_input = $_POST['user_input'];
$password_input = $_POST['password'];

$sql = "SELECT * FROM users WHERE username = :input OR email = :input LIMIT 1";
$stmt = $pdo->prepare($sql);
$stmt->execute(['input' => $user_input]);
$user = $stmt->fetch();

if ($user) {
    $hash = $user['password'];

    // Si la contraseña aún no está hasheada, la convertimos
    if (strlen($hash) < 60 || strpos($hash, '$2y$') !== 0) {
        if ($password_input === $hash) {
            $new_hash = password_hash($password_input, PASSWORD_DEFAULT);
            $update = $pdo->prepare("UPDATE users SET password = ? WHERE id = ?");
            $update->execute([$new_hash, $user['id']]);
            $hash = $new_hash;
        } else {
                    $_SESSION['login_error'] = "Credenciales inválidas. Verifica tu usuario y
contraseña.";
            header("Location: index.php");
            exit;
        }
    }

    // Verificamos la contraseña hasheada
    if (password_verify($password_input, $hash)) {
        $_SESSION['user'] = $user['username'];
        $_SESSION['role'] = $user['role'];
        $_SESSION['user_id'] = $user['id'];

        if ($user['role'] === 'admin') {
            header("Location: admin.php");
        } else {
            header("Location: user.php");
        }
        exit;
    }
```

Conexión a la base de datos

Este bloque establece una conexión con la BD usando PDO **(PHP Data Objects)**

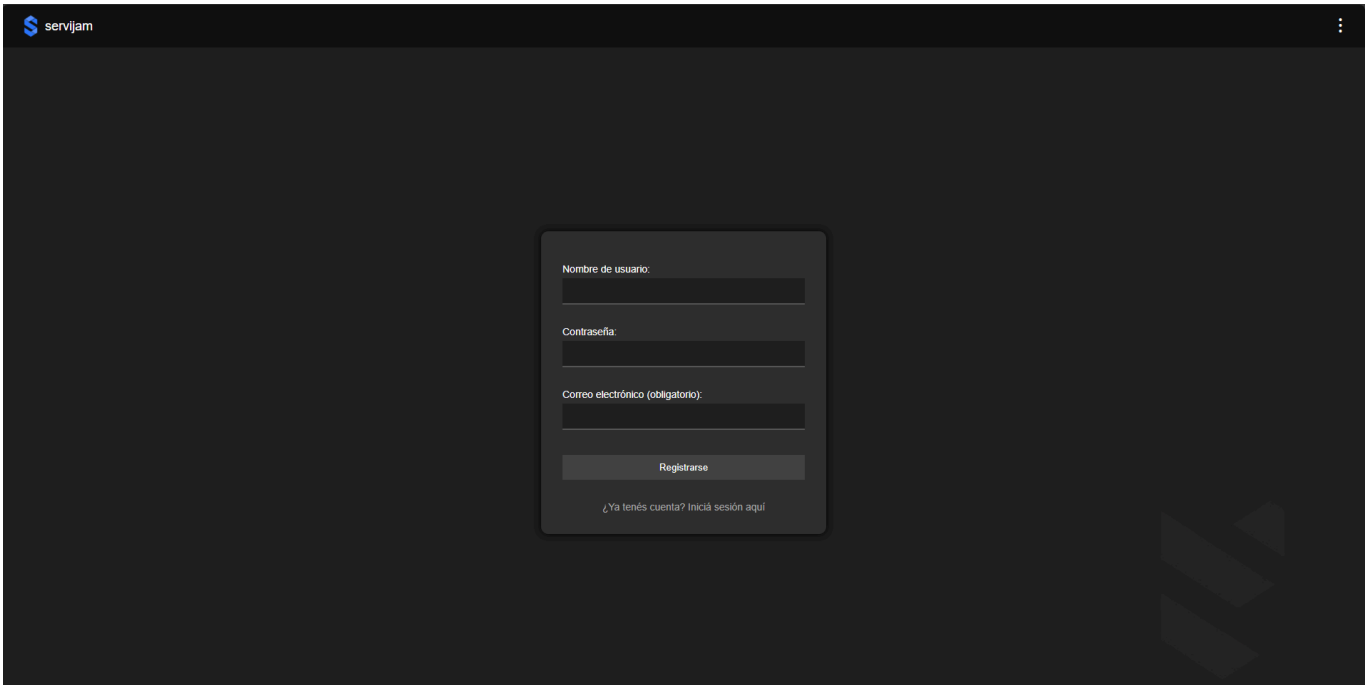
Utiliza un bloque try-catch para manejar errores, dentro del try, crea una instancia de PDO con los parámetros definidos **(host, dbname, username, password)**, especificando la codificación UTF-8 para compatibilidad con caracteres especiales y emojis.

Si ocurre un fallo en la conexión el bloque catch captura el fallo y detiene la ejecución, mostrando un mensaje del error.

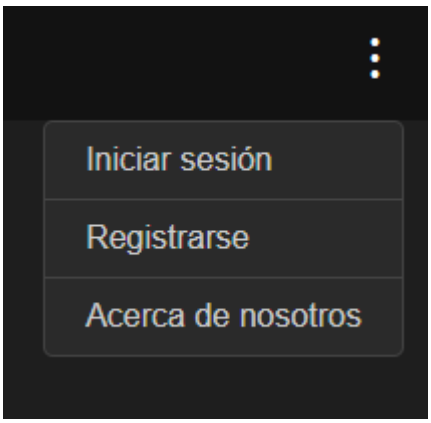
```
<?php
$host = "direcion.ip:puerto";
$dbname = "DataBase";
$username = "username";
$password = "Pa$$w0rd.";

try {
    $pdo = new PDO("mysql:host=$host;dbname=$dbname;charset=utf8mb4", $username, $password);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Error de conexión: " . $e->getMessage());
}
?>
```

Registro.

El recuadro de inicio de sesión cuenta con un enlace en la parte inferior, este enlace redirige a un formulario de registro **(nombre de usuario, contraseña y correo electrónico)**. Estos datos se almacenarán en la base de datos para posteriormente validar las sesiones.

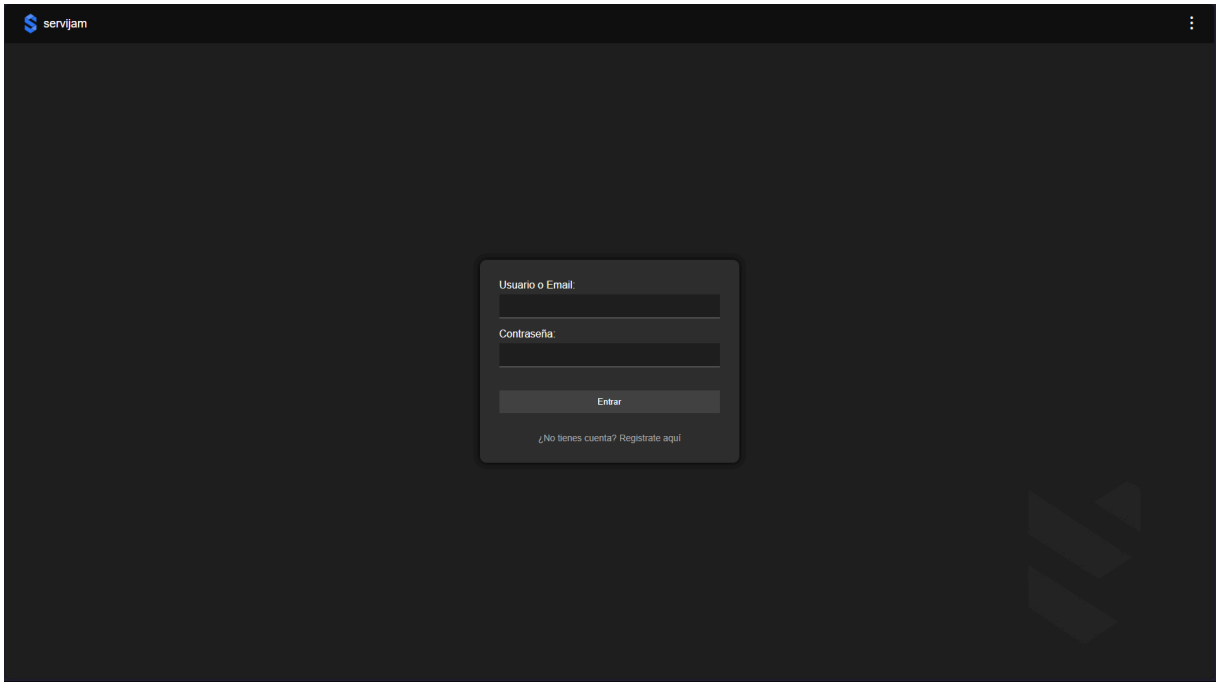


En la parte superior a la derecha contamos con tres puntos, si pinchamos sobre se abrirá un pequeño menú con varias opciones.

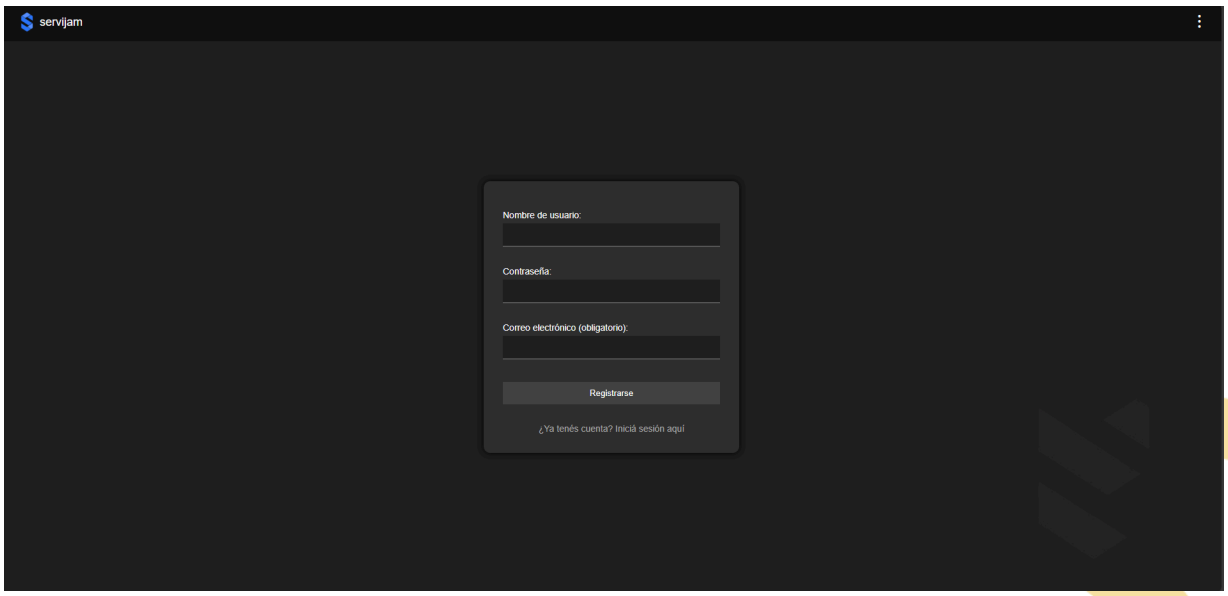


Este menú nos ofrece las siguientes opciones:

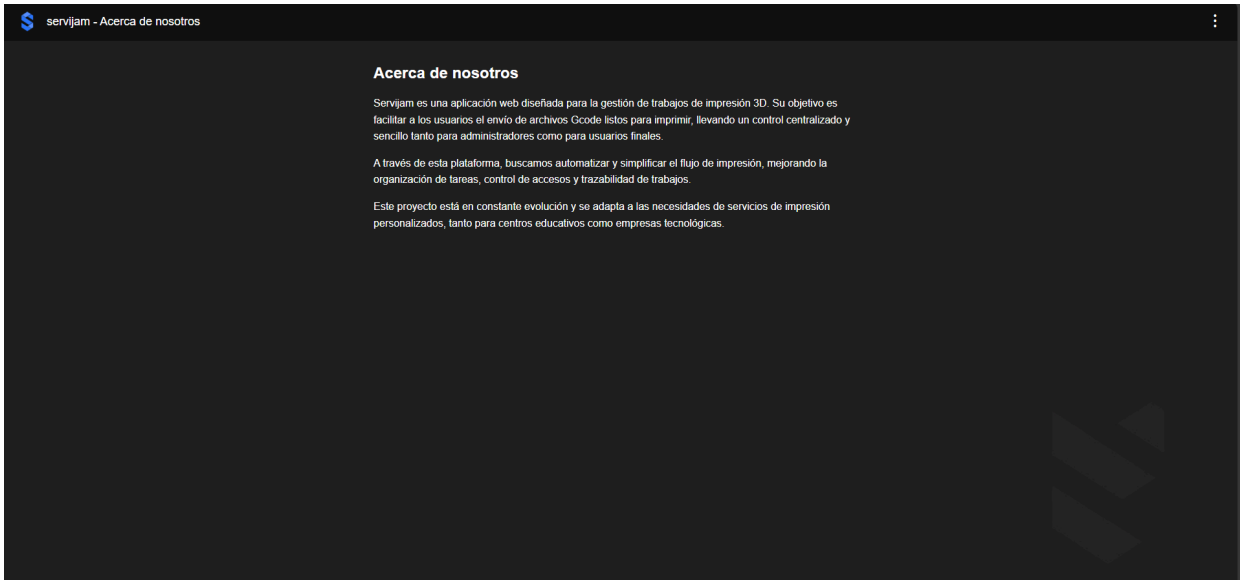
Iniciar sesión. Nos lleva al inicio de sesión.



Registrarse. Nos lleva al formulario de registro.



Acerca de nosotros. Nos lleva a un apartado donde contamos nuestro propósito con **SERVIJAM**.



Código para registro de usuario.

En este código se encuentra el contenedor de registro, que muestra un formulario el cual solicita un nombre de usuario y una contraseña , además de un correo electrónico (**obligatorios todos**). Este formulario envía los datos mediante POST al archivo process_register.php. Al hacer clic en "Registrarse", se envían los datos a la base de datos para crear una cuenta nueva.

```
<div class="topbar">
        <div class="logo">
            <img src="imgs/logo.jpg" alt="Logo">
        </div>
        <div class="title">servijam</div>
        <div class="menu-button" onclick="toggleMenu()">⋮</div>
        <div class="dropdown" id="dropdown-menu">
            <div onclick="location.href='index.php'">Iniciar sesión</div>
            <div onclick="location.href='register.php'">Registrarse</div>
            <div onclick="location.href='about.php'">Acerca de nosotros</div>
        </div>
    </div>


    <div class="login-box">
        <form class="login-form" action="process_register.php" method="post">
            <label>Nombre de usuario:</label>
            <input type="text" name="username" required>

            <label>Contraseña:</label>
            <input type="password" name="password" required>

            <label>Correo electrónico (obligatorio):</label>
            <input type="email" name="email">

            <input type="submit" value="Registrarse">
            <a href="index.php">¿Ya tienes cuenta? Iniciá sesión aquí</a>
        </form>
    </div>
```

Código de registro en la base de datos.

Este código se encarga de procesar el registro de un nuevo usuario primero, establece la conexión a la base de datos. Luego, recoge los datos enviados por el formulario: username, password y email.

A continuación, la contraseña introducida se cifra utilizando un hash.

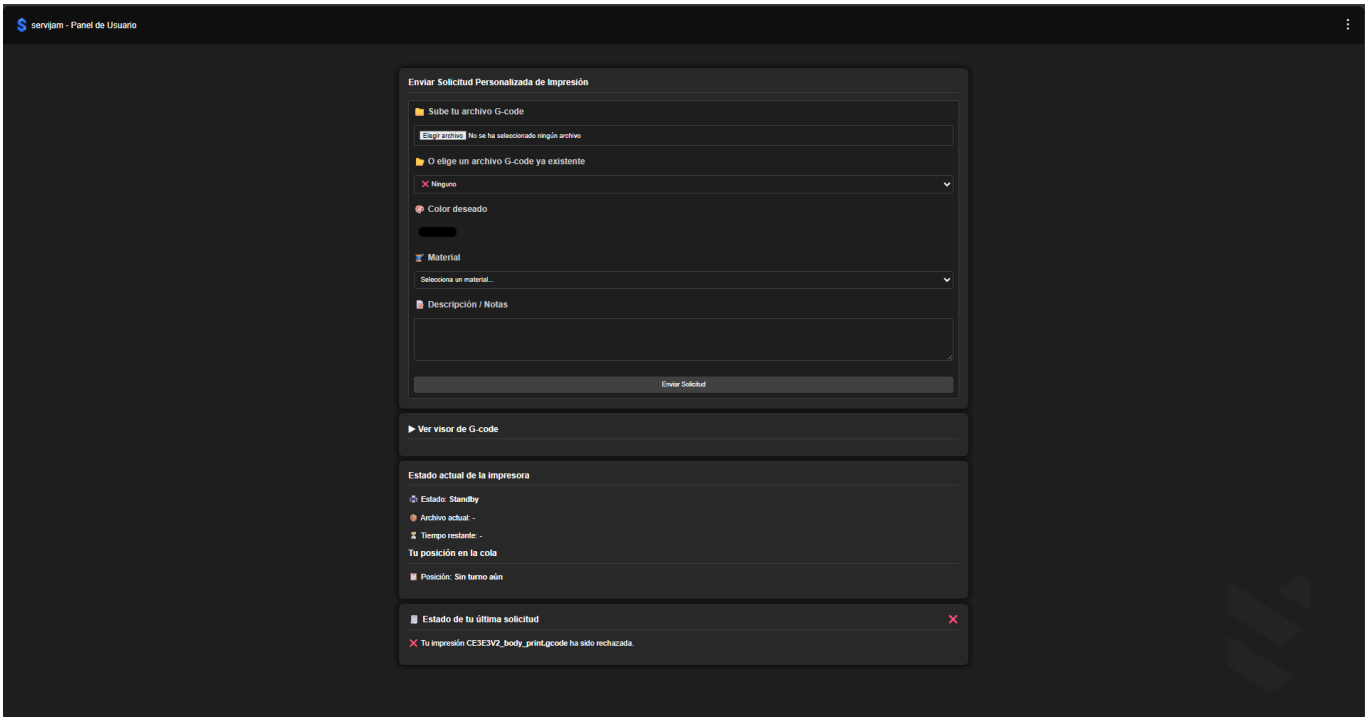
Antes de registrar al usuario, el script comprueba si el nombre de usuario ya existe en la BD. Para ello, hace una consulta (SELECT). Si encuentra alguna coincidencia, detiene el proceso y muestra un mensaje de error.

Si el nombre está libre, ejecuta una consulta INSERT INTO para guardar los datos del nuevo usuario **(nombre, contraseña y correo electrónico)**.

```
require 'db.php';

$username = $_POST['username'];
$password = $_POST['password'];
$email = !empty($_POST['email']) ? $_POST['email'] : null;

$hashed_password = password_hash($password, PASSWORD_DEFAULT);

$stmt = $pdo->prepare("SELECT * FROM users WHERE username = ?");
$stmt->execute([$username]);
if ($stmt->rowCount() > 0) {
    die("El nombre de usuario ya está en uso.");
}

$sql = "INSERT INTO users (username, password, email) VALUES (:username, :password,
:email)";
$stmt = $pdo->prepare($sql);
$stmt->execute([
    'username' => $username,
    'password' => $hashed_password,
    'email' => $email
]);

header("Location: registro_exitoso.php");
exit();
```

Sesión de usuario

La sesión de usuario es una interfaz interactiva que cuenta con cuatro bloques, dichos bloques le ofrecen al usuario la posibilidad de subir sus bocetos y solicitar las impresiones, ver el estado de la impresora y su impresión.




```
<div class="topbar">
    <div class="logo"><img src="imgs/logo.jpg" alt="Logo"></div>
    <div class="title">servijam - Panel de Usuario</div>
    <div class="menu-button" onclick="toggleMenu()">⋮</div>
    <div class="dropdown" id="dropdown-menu">
        <div><a href="about.php">Acerca de nosotros</a></div>
        <div><a href="logout.php">Cerrar sesión</a></div>
    </div>
</div>


<div class="container">
       <div class="panel full-panel">
    <h2>Enviar Solicitud Personalizada de Impresión</h2>
    <form method="post" enctype="multipart/form-data">
        <div class="form-group">
            <label for="file">📁 Sube tu archivo G-code</label>
            <input type="file" id="file" name="file" accept=".gcode">
        </div>
```

```
        <div class="form-group">
              <label for="file_select">📂 O elige un archivo G-code ya existente</label>
              <select id="file_select" name="file_select">
                     <option value="">❌ Ninguno</option>
                     <?php foreach ($gcodeFiles as $file): ?>
                            <option value="<?= htmlspecialchars($file['path']) ?>">
                                   <?= htmlspecialchars($file['path']) ?> (<?=
number_format($file['size'] / 1024, 1) ?> KB)
                            </option>
                     <?php endforeach; ?>
              </select>
        </div>

         <div class="form-group">
             <label for="color">🎨 Color deseado</label>
             <input type="color" id="color" name="color" value="#000000" required
style="height: 40px; width: 100px; border: none; cursor: pointer;">
         </div>

         <div class="form-group">
             <label for="material">🧵 Material</label>
              <select id="material" name="material" required>
                     <option value="" disabled selected>Selecciona un material...</option>
                     <?php foreach ($enum_values as $mat): ?>
                            <option value="<?= htmlspecialchars($mat) ?>"><?=
htmlspecialchars($mat) ?></option>
                     <?php endforeach; ?>
              </select>

         </div>

         <div class="form-group">
             <label for="notes">📝 Descripción / Notas</label>
             <textarea id="notes" name="notes" rows="4"></textarea>
         </div>

         <button type="submit" name="submit_request">Enviar Solicitud</button>
     </form>
</div>



         <div class="panel full-panel">
             <h2 style="cursor:pointer;" onclick="toggleViewer()">▶ Ver visor de G-code</h2>
             <div id="viewerContainer" style="display: none;">
                 <iframe src="viewer/"></iframe>
             </div>
         </div>
```

```
    <div class="panel full-panel">
        <h2>Estado actual de la impresora</h2>
        <p>🖨️ Estado: <strong><?= $printer_status ?></strong></p>
        <p>📦 Archivo actual: <strong><?= $job_filename ?></strong></p>
        <p>⏳ Tiempo restante: <strong><?= $job_remaining ?></strong></p>
        <h2>Tu posición en la cola</h2>
        <p>📋 Posición: <strong><?= $position > 0 ? $position : 'Sin turno aún';
?></strong></p>
    </div>


<?php if ($estado_impresion): ?>
    <div class="panel full-panel">
        <h2>🧾 Estado de tu última solicitud <span style="float:right; cursor:pointer;"
onclick="this.parentElement.parentElement.style.display='none';">❌</span></h2>
        <p><?= $estado_impresion ?></p>
    </div>
```

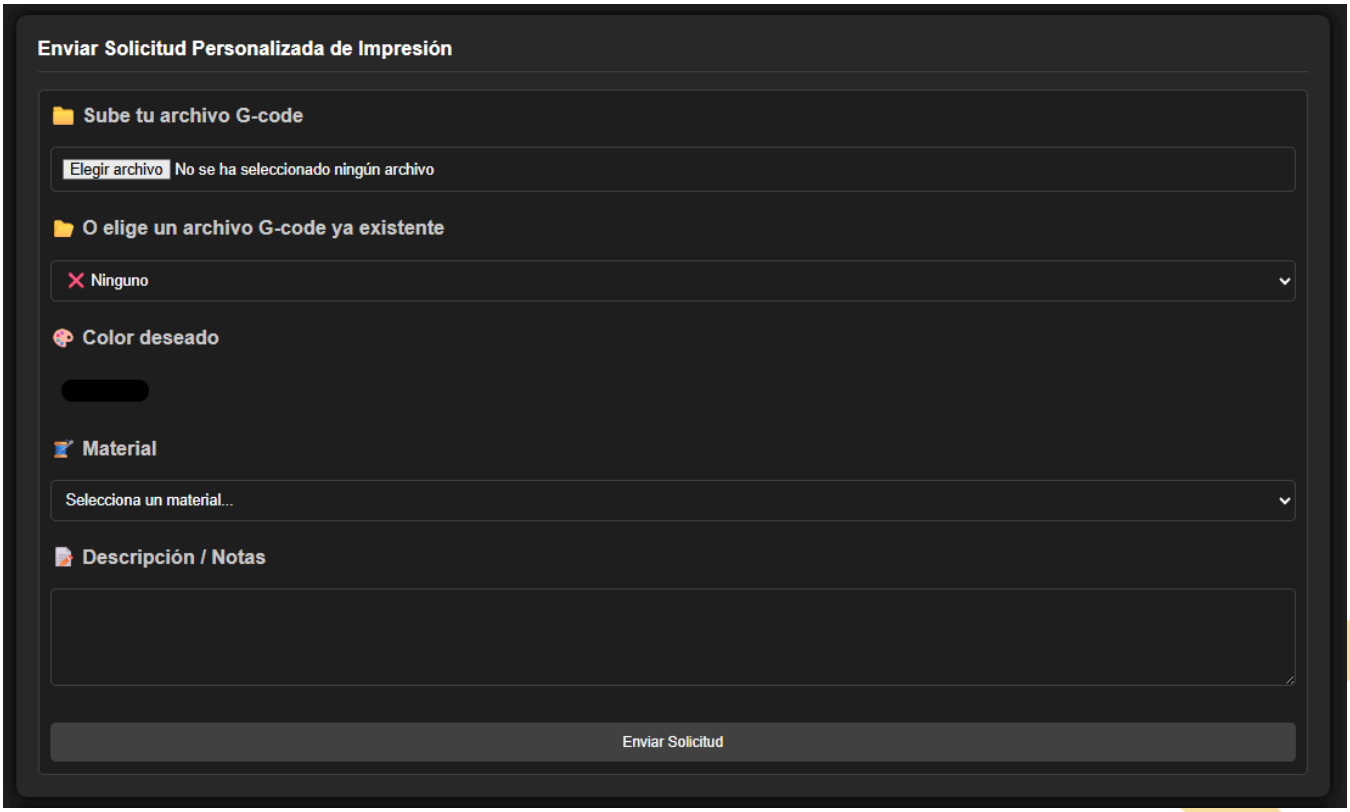
Los mencionados bloques que componen la interfaz de usuario son los siguientes:

- Solicitud de impresión:
  - Función getGcodeFiles():
    Muestra una lista de archivos G-code disponibles para seleccionar. Lo hace mediante una petición cURL a una API y convierte la respuesta en un array PHP.

  - Cuando se envía el formulario **(POST + submit_request)**:
    Se recogen los datos del formulario **(usuario, color, material, notas)** y se obtiene el nombre de usuario y el correo desde la base de datos (usando user_id de sesión).
    Se comprueba si el usuario ha subido un nuevo archivo o seleccionado uno ya existente.

  - Validación de archivo:
    Si el usuario sube un archivo, se comprueba que sea .gcode. Si pasa la validación, se le asigna un identificador único y se guarda.
    Si elige un archivo existente, se usa directamente el nombre del archivo.

    Si intenta hacer ambas cosas a la vez o ninguna, el sistema lanza un error.



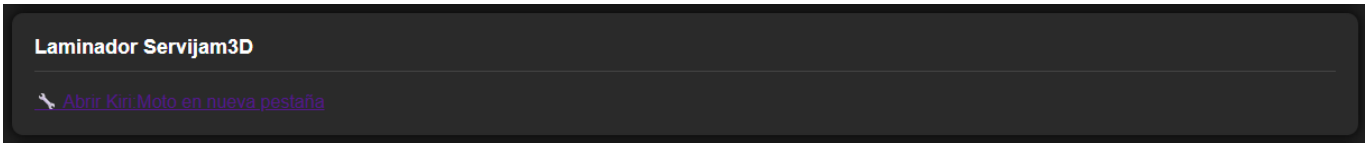

```
function getGcodeFiles() {
    $url = "http://100.121.129.41:7125/server/files/list?root=gcodes";
    $ch = curl_init($url);
    curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);
    $response = curl_exec($ch);
    curl_close($ch);
    $data = json_decode($response, true);
    return $data['result'] ?? [];
```

```
}
$gcodeFiles = getGcodeFiles();

if ($_SERVER['REQUEST_METHOD'] === 'POST' && isset($_POST['submit_request'])) {
    $user_id = $_SESSION['user_id'];
    $color = $_POST['color'];
    $material = $_POST['material'];
    $notes = $_POST['notes'];
    $upload_dir = __DIR__ . '/uploads/';
    $filename = '';



    // Obtener datos del usuario
    $stmt = $pdo->prepare("SELECT username, email FROM users WHERE id = ?");
    $stmt->execute([$user_id]);
    $user = $stmt->fetch(PDO::FETCH_ASSOC);

    // Verificar si se subió archivo o se seleccionó uno existente
    $archivo_subido = !empty($_FILES['file']['name']);
    $archivo_seleccionado = !empty($_POST['file_select']);

$error = '';

if ($archivo_subido && $archivo_seleccionado) {
    $error = "Por favor, elige solo una opción: subir un archivo o seleccionar uno
existente.";
} elseif (!$archivo_subido && !$archivo_seleccionado) {
    $error = "Debes subir un archivo o seleccionar uno existente.";
} elseif ($archivo_subido) {
    $file_ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
    if ($_FILES['file']['error'] === UPLOAD_ERR_OK && $file_ext === 'gcode') {
        $filename = uniqid('job_', true) . '.gcode';
        move_uploaded_file($_FILES['file']['tmp_name'], $upload_dir . $filename);
    } else {
        $error = "Solo se permiten archivos .gcode.";
    }
}

if (!$archivo_subido && $archivo_seleccionado) {
    $filename = $_POST['file_select'];
}

if (!empty($error)) {
    $_SESSION['form_error'] = $error;
    header("Location: " . $_SERVER['PHP_SELF']);
    exit;
}

    // Validar archivo subido
    if ($archivo_subido) {
        $file_ext = strtolower(pathinfo($_FILES['file']['name'], PATHINFO_EXTENSION));
        if ($_FILES['file']['error'] === UPLOAD_ERR_OK && $file_ext === 'gcode') {
            // Renombrar el archivo para evitar colisiones
            $filename = uniqid('job_', true) . '.gcode';
            move_uploaded_file($_FILES['file']['tmp_name'], $upload_dir . $filename);
        } else {
            die("❌ Solo se permiten archivos .gcode.");
        }
    }

    // Si eligió archivo existente
    if (!$archivo_subido && $archivo_seleccionado) {
```

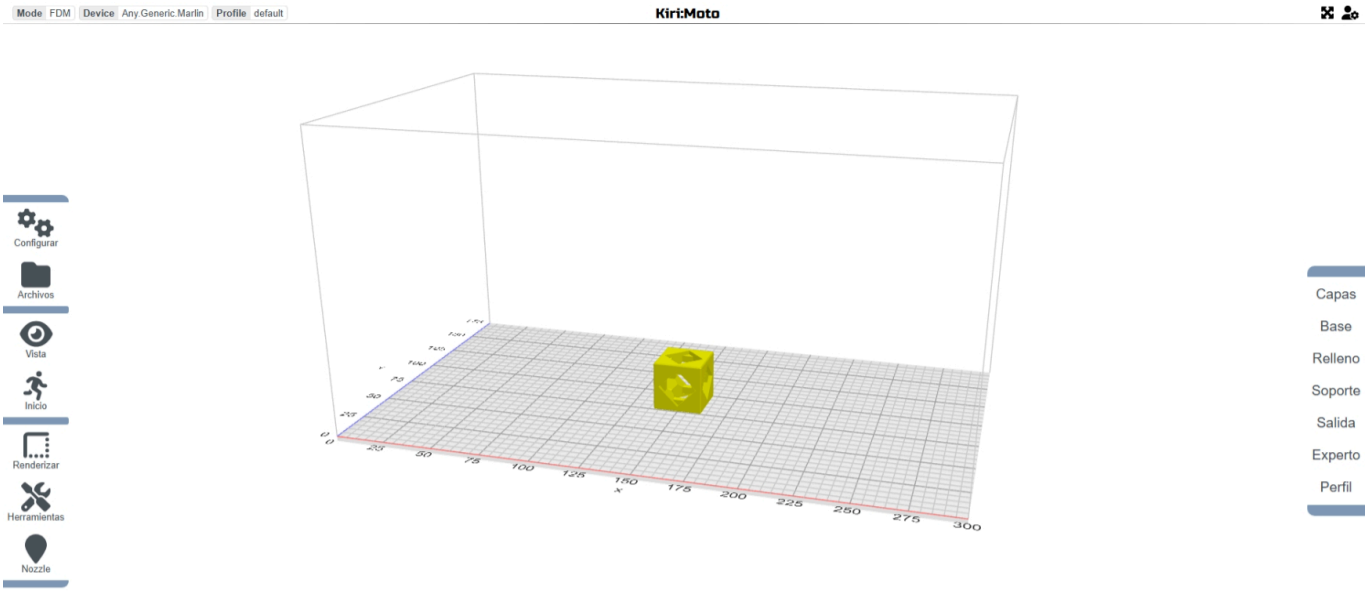
```
        $filename = $_POST['file_select'];
    }
```

- Laminador:

Kiri:Moto es un laminador que permite preparar archivos 3D para impresión 3D, corte láser y mecanizado CNC. Su funcionamiento se basa en la segmentación del modelo tridimensional (.STL, .OBJ, entre otros) en capas horizontales, generando posteriormente un archivo G-code optimizado que define las instrucciones precisas que debe seguir la máquina durante el proceso de fabricación.



Adicionalmente



Adicionalmente de la configuraciones básicas con **Kiri:Moto** tiene algunas configuraciones avanzadas como:

Distancia de Retracción:
Indica cuántos milímetros de filamento se retraen del hotend cuando la boquilla se desplaza sin imprimir, para evitar goteo.

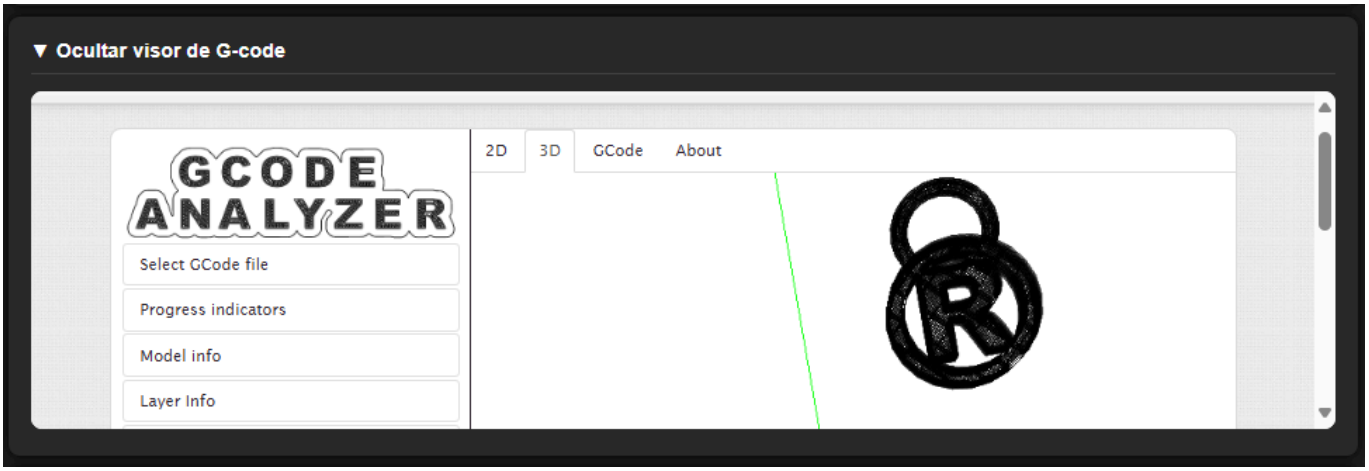
Retracción de Barrido:
No activa un movimiento de limpieza lateral de la boquilla tras la retracción. Un valor mayor podría ayudar a evitar acumulaciones de material.

Activar el Reposo (20):
Define un tiempo de reposo tras la retracción antes de continuar la impresión. Este valor mejora la estabilidad del material extraído.

Ruta Corta:
Prioriza trayectorias más cortas entre movimientos, lo que reduce tiempos de impresión y posibles defectos por desplazamientos largos.

Distancia Z de Salto:
Levanta ligeramente el eje Z durante desplazamientos sin impresión (z-hop), útil para evitar colisiones con el modelo ya impreso.

- Visor de G-code Gcode Analyzer:
  - Desde este bloque obtendremos una vista previa del archivo G-code.



  - Además de una vista previa la herramienta tiene varios apartados como; "**Model info**" el cual nos da información sobre la impresión.

Model info

Model size is:
155.00x38.59x73.24mm
Total filament used: 604.60mm
Total filament weight used: 1.51grams
Estimated print time: 0:11:17
Estimated layer height: 3.66mm
Layer count: 21printed, 25visited
Time cost: 0.19
Filament cost: 0.08

"**Layer info**" nos muestra información relevante sobre las capas que se imprimen, datos como: número de capa, filamento usado por capa, tiempo por capa y la velocidad en mm/s tanto del extrusor como de los movimientos de los ejes.

Layer Info

Layer number: 0
Layer height (mm): 0.2
GCODE commands in layer: 4
Filament used by layer (mm): 15.00
Filament used, summed (mm): 15.00
Print time for layer: 4.5sec
Extrude speeds:
= 10.00mm/s
Move speeds:
= 83.33mm/s
= 80.00mm/s
= 50.00mm/s
= 10.00mm/s

"**2D Render Options**" en este apartado podemos seleccionar ¿?

Speed display type:

- (●) mm/sec
- ( ) mm extrusion per mm move
- ( ) mm^3/sec
- [x] Show non-extrusion moves
- [x] Show retracts and restarts
- [x] Move model to the center of the grid
- [x] Show different speeds with different colors
- [ ] Emulate extrusion width
- [ ] Render lines slightly transparent
- [ ] Show +1 layer

"GCode analyzer options" en este apartado podemos seleccionar ¿?

GCode analyzer options

These require re-analyzing file:

- [x] Sort layers by Z
- [x] Hide empty layers
- [x] Show GCode in GCode tab (memory intensive!)

"Printer Info" aquí podremos configurar y ver información sobre la bobina de filamento introducida en la impresora, podemos ver información como: tipo de plástico, el tamaño del nozzle, el diámetro del hilo de la bobina de filamento y el costo por hora.

Plastic diameter:

1.75

Plastic type:

- (●) ABS
- ( ) PLA

Nozzle size:

0.4

- [ ] Volumetric E

Printer hourly cost:

1.00

- Estado actual de la impresora.
  Este bloque consulta el estado en tiempo real de la impresora 3D usando una API externa **(Moonraker)**.

  Hace una petición (file_get_contents) a una URL local (/printer/objects/query) para obtener:
  - El estado de la impresora **(printing, ready, paused...)**.
  - El nombre del archivo que se está imprimiendo.
  - Duración total estimada y cuánto lleva impreso.

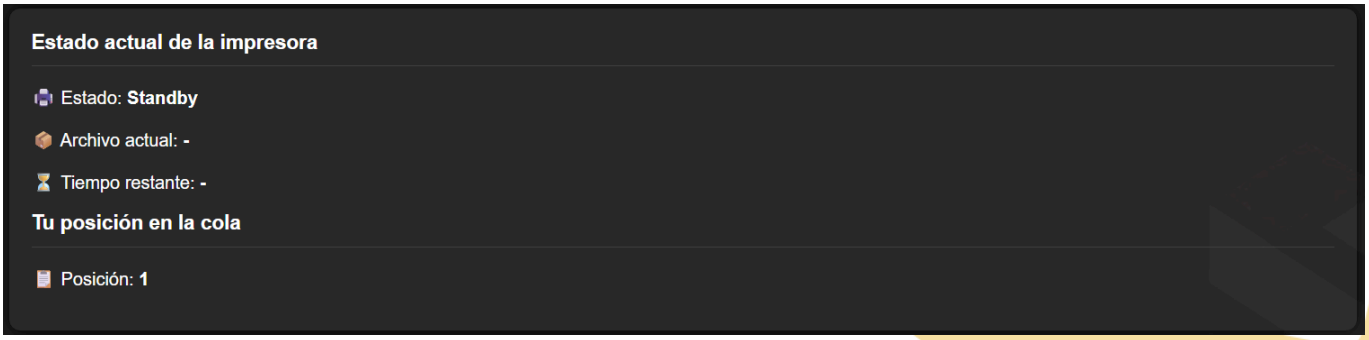
  Al obtener respuesta, se analiza el contenido y se extrae el estado actual de impresión.

  Dependiendo del estado, si está imprimiendo:
  - Muestra el nombre del archivo en curso.
  - Calcula el tiempo restante: se resta el tiempo ya impreso al total estimado.

  Si está lista:
  - Dice que está “Lista” y esperando trabajo.

  Si está pausada:
  - Informa que la impresión está pausada.

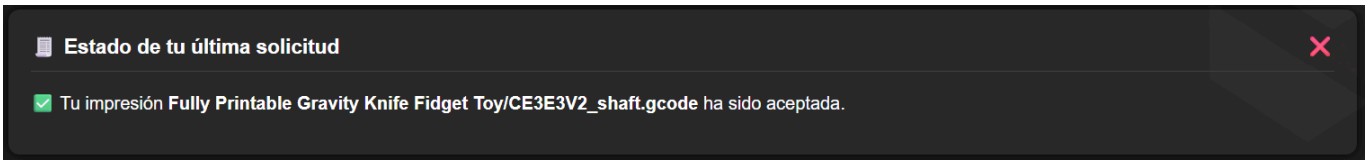
  Otros estados desconocidos:
  - Se muestran tal cual, con valores genéricos.



```
// Estado de impresora
$printer_status = "Desconocido";
$job_filename = "-";
$job_remaining = "-";

$api_url =
'http://100.121.129.41:7125/printer/objects/query?print_stats=filename,state,print_duration,to
tal_duration';
$response = @file_get_contents($api_url);
if ($response !== FALSE) {
    $data = json_decode($response, true);
    $status = $data['result']['status']['print_stats']['state'] ?? 'unknown';

    switch ($status) {
        case 'printing':
            $printer_status = 'Imprimiendo';
            $job_filename = $data['result']['status']['print_stats']['filename'] ?? '-';
            $job_remaining = round($data['result']['status']['print_stats']['total_duration']
- $data['result']['status']['print_stats']['print_duration']) . " s";
            break;
        case 'ready':
            $printer_status = 'Lista';
            $job_filename = '-';
            $job_remaining = 'Esperando trabajo';
            break;
        case 'paused':
            $printer_status = 'Pausada';
            $job_remaining = 'Impresión pausada';
            break;
        default:
            $printer_status = ucfirst($status);
            $job_filename = '-';
            $job_remaining = '-';
            break;
    }
}
```

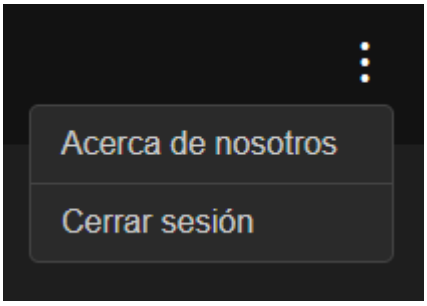
- Estado de la solicitud:
  - Aquí el cliente podrá ver el estado de su solicitud, si ha sido aprobada/rechazada o si está llevando a cabo la impresión o si aun sigue en cola.

    Primero comprueba si la variable $estado_impresion tiene contenido. Si lo tiene, genera un panel en la interfaz con el mensaje.



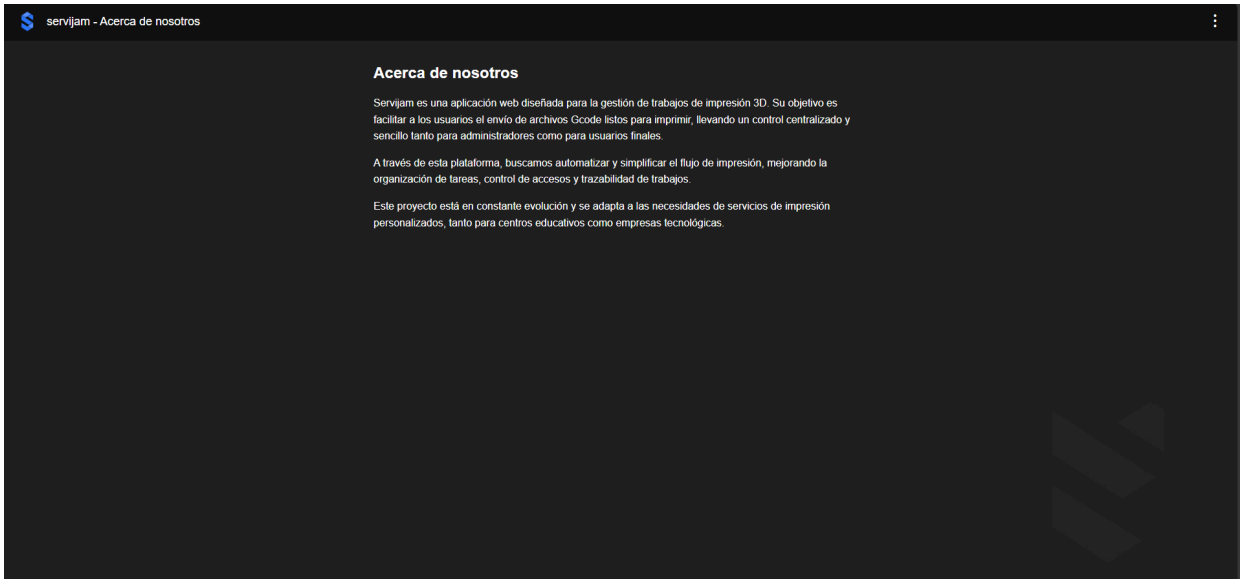

```
<?php if ($estado_impresion): ?>
    <div class="panel full-panel">
        <h2>🧾 Estado de tu última solicitud <span style="float:right; cursor:pointer;"
onclick="this.parentElement.parentElement.style.display='none';">❌</span></h2>
        <p><?= $estado_impresion ?></p>
    </div>
<?php endif; ?>
```

En la parte superior a la derecha contamos con tres puntos, si pinchamos sobre se abrirá un pequeño menú con dos opciones.

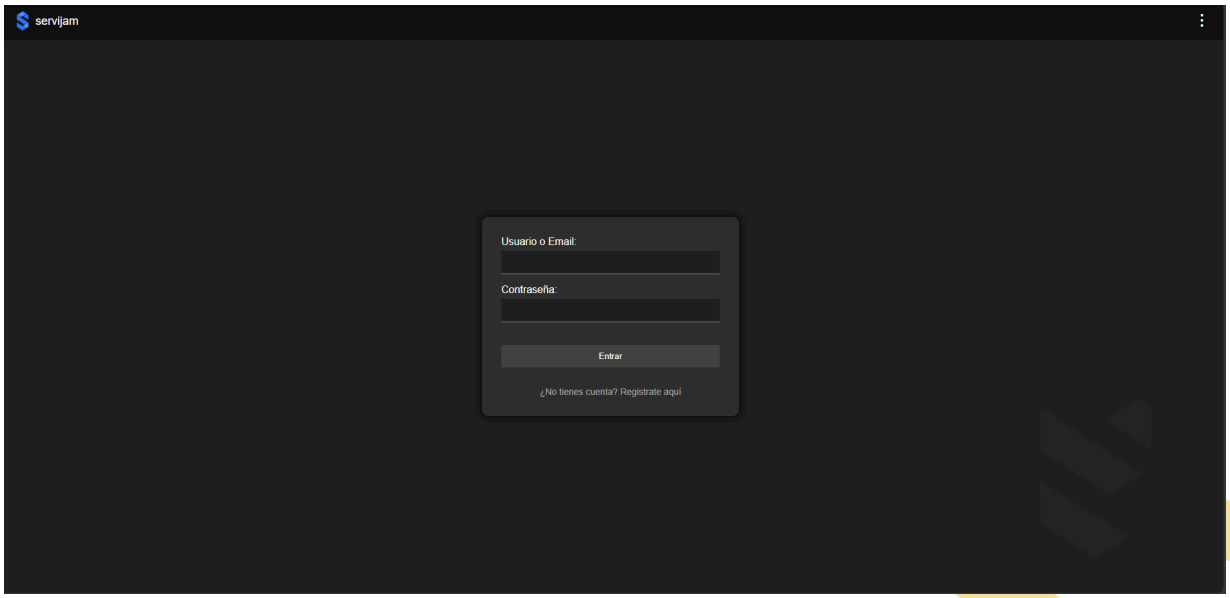


Este menú nos ofrece las siguientes opciones:

Acerca de nosotros. Nos lleva a un apartado donde contamos nuestro propósito con **SERVIJAM**.

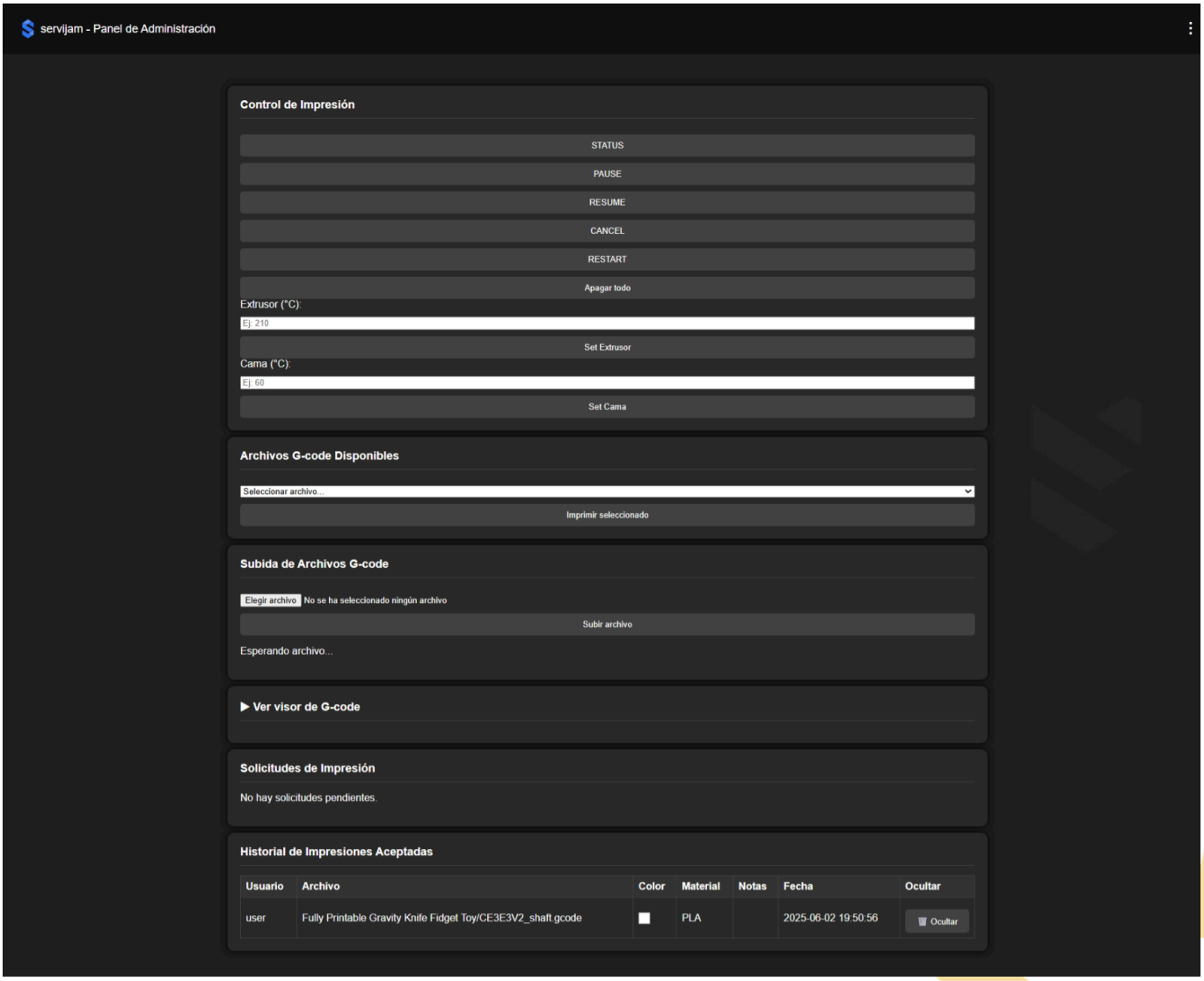


Cerrar sesión. Cierra la sesión actual **(usuario/administrador)** y redirige al inicio de sesión.



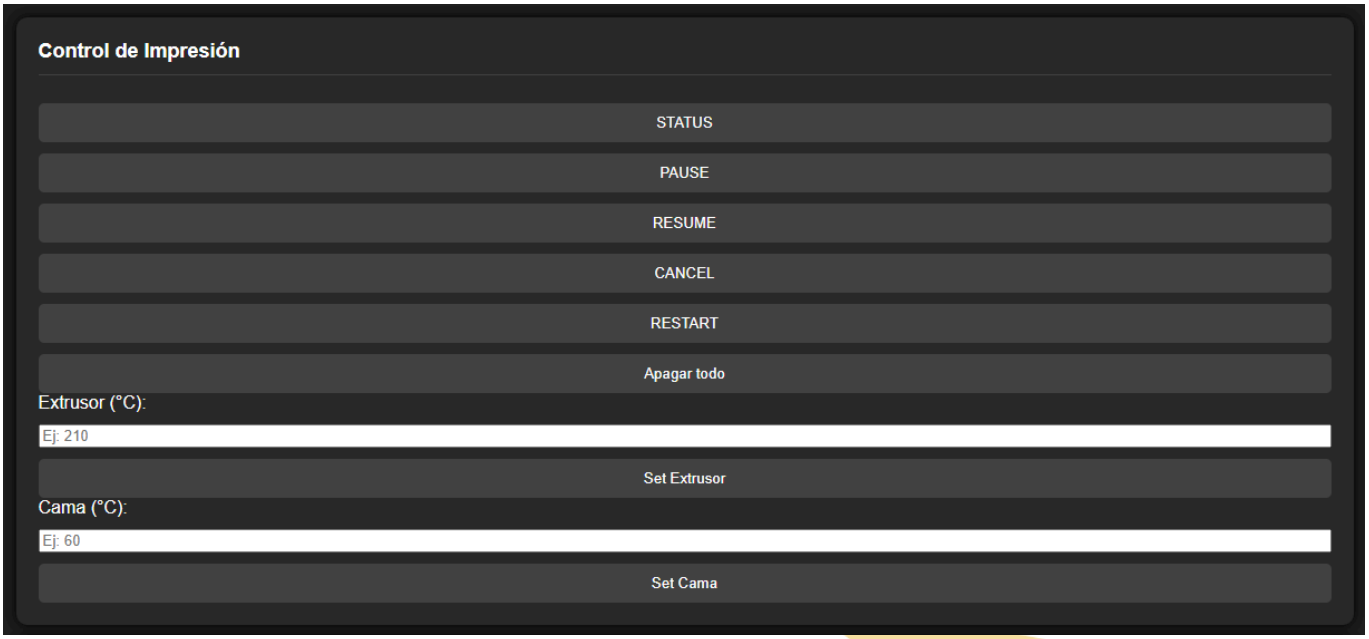
Sesión de administrador

La sesión de administrador es una interfaz interactiva que cuenta con seis bloques, algunos de los bloques cumplen las mismas funciones que algunos de los paneles de usuario, sin embargo cuenta con otros paneles extra que permiten al administrador controlar ciertas funciones de la impresora y de los procesos. Además de los controles sobre las solicitudes de impresión



- Control de impresión.
  - En este bloque el administrador tiene control sobre las impresiones que se están llevando a cabo. Cuando se pulsa un botón, el servidor recibe el comando y lo gestiona a través de la función sendCommand(), que se encarga de enviarlo a la impresora.

    - **Status**. Este botón muestra el estado actual de la impresión.
    - **Pause**. Este botón pausa la impresión actual.
    - **Resume**. Este botón reanuda la impresión pausada.
    - **Cancel**. Este botón cancela la impresión.
    - **Restart**. Este botón reinicia la impresión.
    - **Apagar todo**. Este botón apaga la impresora.
    - **Set Extrusor**. Este botón modifica la temperatura del extrusor
    - **Set Cama**. Este botón modifica la temperatura de la cama



```
<div class="panel full-panel">
            <h2>Control de Impresión</h2>
            <form method="post">
                <button name="command" value="STATUS">STATUS</button>
                <button name="command" value="PAUSE">PAUSE</button>
                <button name="command" value="RESUME">RESUME</button>
                <button name="command" value="CANCEL_PRINT">CANCEL</button>
                <button name="command" value="RESTART">RESTART</button>
                <button name="command" value="TURN_OFF_HEATERS">Apagar todo</button>
                <label>Extrusor (°C):</label>
                <input type="number" name="custom_temp_extruder" placeholder="Ej: 210">
                <button name="command" value="set_custom_temp_extruder">Set Extrusor</button>
                <label>Cama (°C):</label>
                <input type="number" name="custom_temp_bed" placeholder="Ej: 60">
                <button name="command" value="set_custom_temp_bed">Set Cama</button>
            </form>
        </div
```

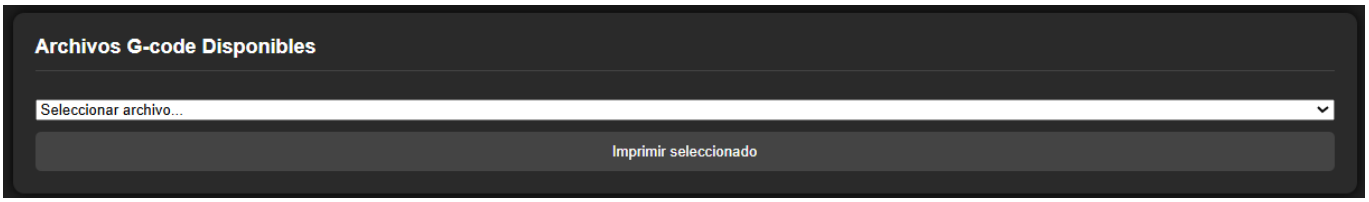
```
if ($_SERVER["REQUEST_METHOD"] === "POST" && isset($_POST['command'])) {
    $cmd = $_POST['command'];

    switch ($cmd) {
        case "STATUS":
        case "PAUSE":
        case "RESUME":
        case "CANCEL_PRINT":
        case "RESTART":
        case "TURN_OFF_HEATERS":
            sendCommand($cmd);
            break;
```

- Archivos G-code disponibles.
  - En este bloque obtendremos un listado donde se podrá seleccionar uno de los G-code que ofrecemos para imprimir.

    El código genera un desplegable **(<select>)** muestra los archivos G-code disponibles, gracias al array $gcodeFiles.

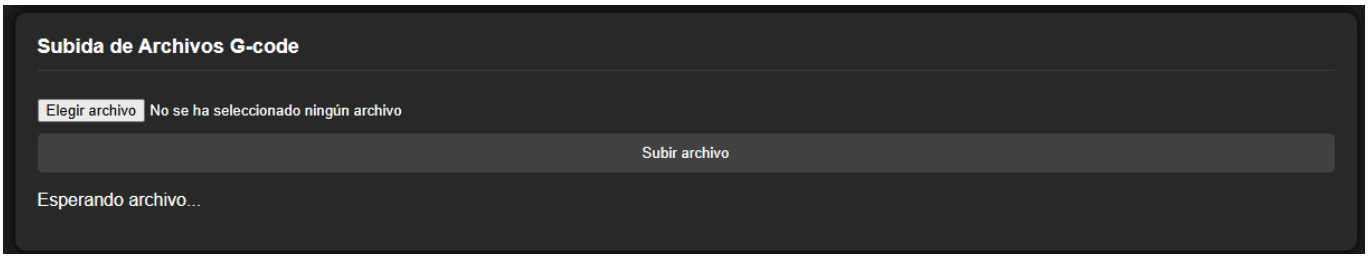
    Cada archivo se muestra como una opción **(<option>)** indicando su ruta **(nombre)** y su tamaño en KB.

    Al pulsar el botón "Imprimir seleccionado", se envía el formulario por POST con el nombre del archivo y el comando print_gcode.



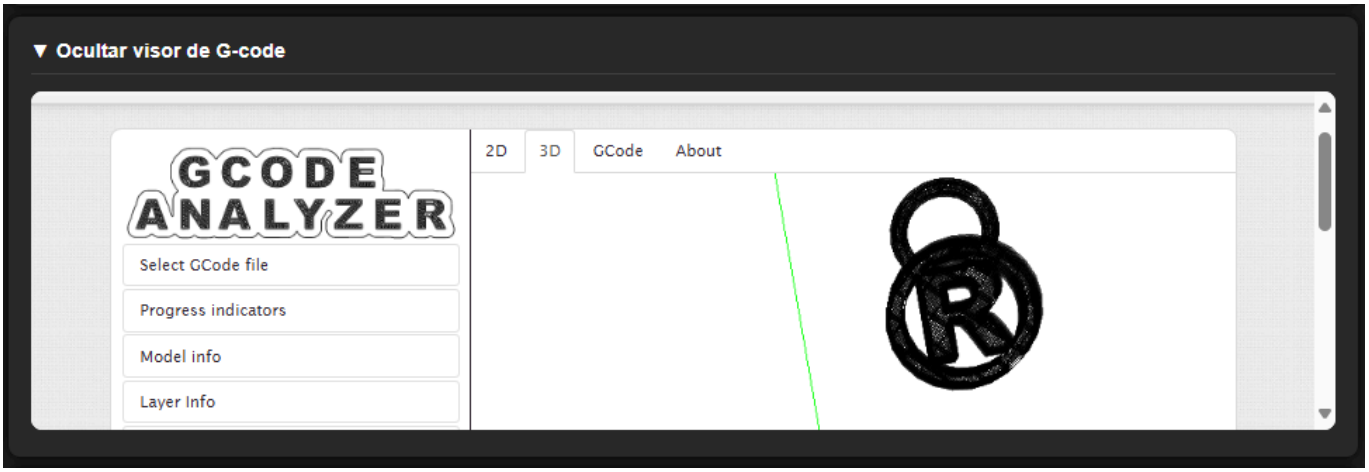

```
<div class="panel full-panel">
            <h2>Archivos G-code Disponibles</h2>
            <form method="post">
                <select name="file">
                    <option disabled selected>Seleccionar archivo...</option>
                    <?php foreach ($gcodeFiles as $file): ?>
                        <option value="<?= htmlspecialchars($file['path']) ?>">
                            <?= htmlspecialchars($file['path']) ?> (<?=
number_format($file['size'] / 1024, 1) ?> KB)
                        </option>
                    <?php endforeach; ?>
                </select>
                <button name="command" value="print_gcode">Imprimir seleccionado</button>
            </form>
        </div>
```

- Subida de archivos G-code.
  - Este bloque crea un formulario para la subida de los archivos G-codes.
  - El <input type="file"> permite al administrador elegir el archivo.
  - Al enviar el formulario, se lanza una función que envía el archivo al servidor, el cual lo guarda en una carpeta como gcode.
  - El párrafo con id="uploadStatus" muestra el mensaje de espera o errores si algo falla.




```
<div class="panel full-panel">
      <h2>Subida de Archivos G-code</h2>
      <form id="uploadForm">
          <input type="file" name="file" required>
          <button type="submit">Subir archivo</button>
      </form>
      <p id="uploadStatus">Esperando archivo...</p>
  </div>

  <div class="panel full-panel">
      <h2 style="cursor:pointer;" onclick="toggleViewer()">▶ Ver visor de G-code</h2>
      <div id="viewerContainer" style="display: none;">
          <iframe src="/viewer/"></iframe>
      </div>
  </div>
```

- Visor de G-code Gcode Analyzer:
  - Desde este bloque obtendremos una vista previa del archivo G-code.



  - Además de una vista previa la herramienta tiene varios apartados como; "**Model info**" el cual nos da información sobre la impresión.

Model info

Model size is:
155.00x38.59x73.24mm
Total filament used: 604.60mm
Total filament weight used: 1.51grams
Estimated print time: 0:11:17
Estimated layer height: 3.66mm
Layer count: 21printed, 25visited
Time cost: 0.19
Filament cost: 0.08

"**Layer info**" nos muestra información relevante sobre las capas que se imprimen, datos como: número de capa, filamento usado por capa, tiempo por capa y la velocidad en mm/s tanto del extrusor como de los movimientos de los ejes.

Layer Info

Layer number: 0
Layer height (mm): 0.2
GCODE commands in layer: 4
Filament used by layer (mm): 15.00
Filament used, summed (mm): 15.00
Print time for layer: 4.5sec
Extrude speeds:
= 10.00mm/s
Move speeds:
= 83.33mm/s
= 80.00mm/s
= 50.00mm/s
= 10.00mm/s

"**2D Render Options**" en este apartado podemos seleccionar ¿?

Speed display type:

- (●) mm/sec
- ( ) mm extrusion per mm move
- ( ) mm^3/sec
- [x] Show non-extrusion moves
- [x] Show retracts and restarts
- [x] Move model to the center of the grid
- [x] Show different speeds with different colors
- [ ] Emulate extrusion width
- [ ] Render lines slightly transparent
- [ ] Show +1 layer

"GCode analyzer options" en este apartado podemos seleccionar ¿?

GCode analyzer options

These require re-analyzing file:

- [x] Sort layers by Z
- [x] Hide empty layers
- [x] Show GCode in GCode tab (memory intensive!)

"Printer Info" aquí podremos configurar y ver información sobre la bobina de filamento introducida en la impresora, podemos ver información como: tipo de plástico, el tamaño del nozzle, el diámetro del hilo de la bobina de filamento y el costo por hora.

Plastic diameter:

1.75

Plastic type:

- (●) ABS
- ( ) PLA

Nozzle size:

0.4

- [ ] Volumetric E

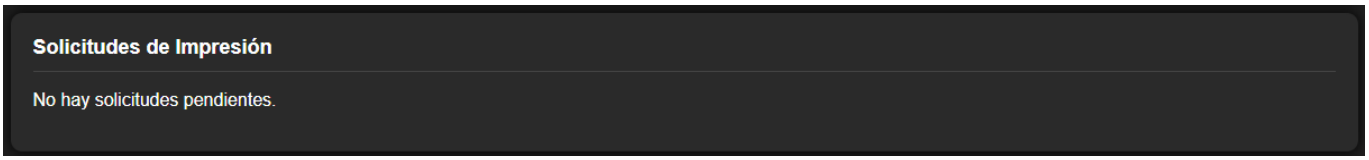
Printer hourly cost:

1.00

- Solicitudes de impresión.
  Desde este bloque el administrador podrá gestionar las solicitudes de impresión enviadas por los usuarios. Si el array $solicitudes contiene elementos, se muestra una tabla con los datos de la solicitud. Si no, se muestra un mensaje que indica que no hay solicitudes pendientes.

  La tabla contiene los datos de cada solicitud:
  - Usuario que envió la solicitud.
  - Nombre del archivo G-code solicitado.
  - Color elegido para la impresión.
  - Material seleccionado.
  - Notas adicionales que el usuario haya dejado.
  -

  Acciones del administrador:
  - Botón para aceptar la solicitud.
  - Otro para rechazarla.



```
<div class="panel full-panel">
      <div class="responsive-table table">
            <h2>Solicitudes de Impresión</h2>
            <?php if (count($solicitudes) > 0): ?>
            <table>
                <thead>
                    <tr>
                        <th>Usuario</th>
                        <th>Archivo</th>
                        <th>Color</th>
                        <th>Material</th>
                        <th>Notas</th>
                        <th>Fecha</th>
                        <th>Acciones</th>
                    </tr>
                </thead>
                <tbody>
                    <?php foreach ($solicitudes as $s): ?>
                        <tr>
                            <td><?= htmlspecialchars($s['username']) ?></td>
                            <td><?= htmlspecialchars($s['filename']) ?></td>
                            <td><?= htmlspecialchars($s['color']) ?></td>
                            <td><?= htmlspecialchars($s['material']) ?></td>
                            <td><?= nl2br(htmlspecialchars($s['notes'])) ?></td>
                            <td><?= htmlspecialchars($s['submitted_at']) ?></td>
                            <td>
                                <form method="post" style="display:inline;">
                                    <button type="submit" name="aceptar" value="<?= $s['id']
?>">Aceptar</button>
                                </form>
                                <form method="post" style="display:inline;">
                                    <button type="submit" name="rechazar" value="<?= $s['id']
?>">Rechazar</button>
                                </form>
                            </td>
                        </tr>
                    <?php endforeach; ?>
                </tbody>
            </table>
            <?php else: ?>
                <p>No hay solicitudes pendientes.</p>
            <?php endif; ?>
        </div>
      </div>
```

- Historial de impresiones aceptadas.
  En este bloque, el administrador podrá observar el historial de todas aquellas solicitudes de impresión que hayan sido aceptadas. Además de una opción para ocultar aquellas entradas que ya no considere necesarias mostrar, sin eliminar permanentemente los registros de la base de datos. Si el array $impresiones_aceptadas contiene datos, se genera una tabla con los registros. Si está vacío, se muestra un mensaje indicando que no hay entradas visibles.

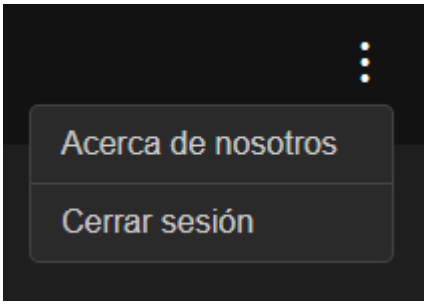
  Se muestran los siguientes datos para cada impresión:
  - Usuario que la envió.
  - Nombre del archivo G-code.
  - Color seleccionado.
  - Material usado.
  - Notas adicionales.
  - Fecha de solicitud.

Historial de Impresiones Aceptadas

Usuario	Archivo	Color	Material	Notas	Fecha	Ocultar
user	Fully Printable Gravity Knife Fidget Toy/CE3E3V2_shaft.gcode		PLA		2025-06-02 19:50:56	Ocultar

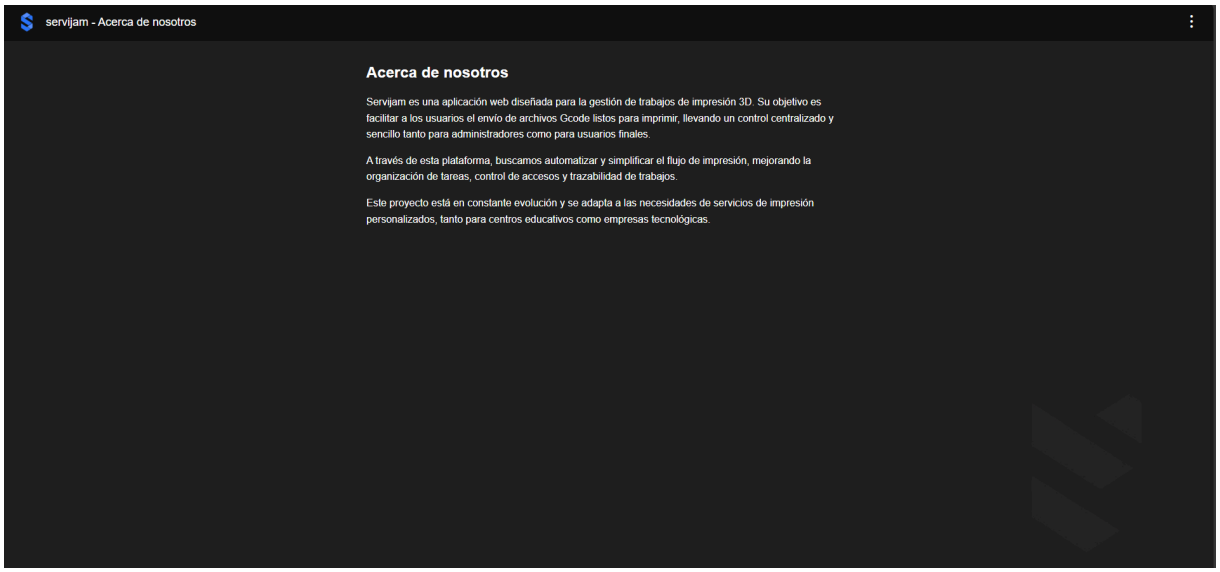
```
<div class="panel full-panel">
      <div class="responsive-table table">
            <h2>Historial de Impresiones Aceptadas</h2>
            <table>
                <thead>
                    <tr>
                        <th>Usuario</th>
                        <th>Archivo</th>
                        <th>Color</th>
                        <th>Material</th>
                        <th>Notas</th>
                        <th>Fecha</th>
                        <th>Ocultar</th>
                    </tr>
                </thead>
                <tbody>
                    <?php if (!empty($impresiones_aceptadas)) : ?>
                        <?php foreach ($impresiones_aceptadas as $imp): ?>
                            <tr>
                                <td><?= htmlspecialchars($imp['username']) ?></td>
                                <td><?= htmlspecialchars($imp['filename']) ?></td>
                                <td><?= htmlspecialchars($imp['color']) ?></td>
                                <td><?= htmlspecialchars($imp['material']) ?></td>
                                <td><?= nl2br(htmlspecialchars($imp['notes'])) ?></td>
                                <td><?= htmlspecialchars($imp['submitted_at']) ?></td>
                                <td>
                                    <form method="post" style="display:inline;">
                                        <button type="submit" name="ocultar_historial"
value="<?= $imp['id'] ?>">🗑️ Ocultar</button>
                                    </form>
                                </td>
                            </tr>
                        <?php endforeach; ?>
                    <?php else: ?>
                        <tr><td colspan="7">No hay impresiones aceptadas visibles.</td></tr>
                    <?php endif; ?>
                </tbody>
            </table>
        </div>
      </div>
```

En la parte superior a la derecha contamos con tres puntos, si pinchamos sobre se abrirá un pequeño menú con dos opciones.

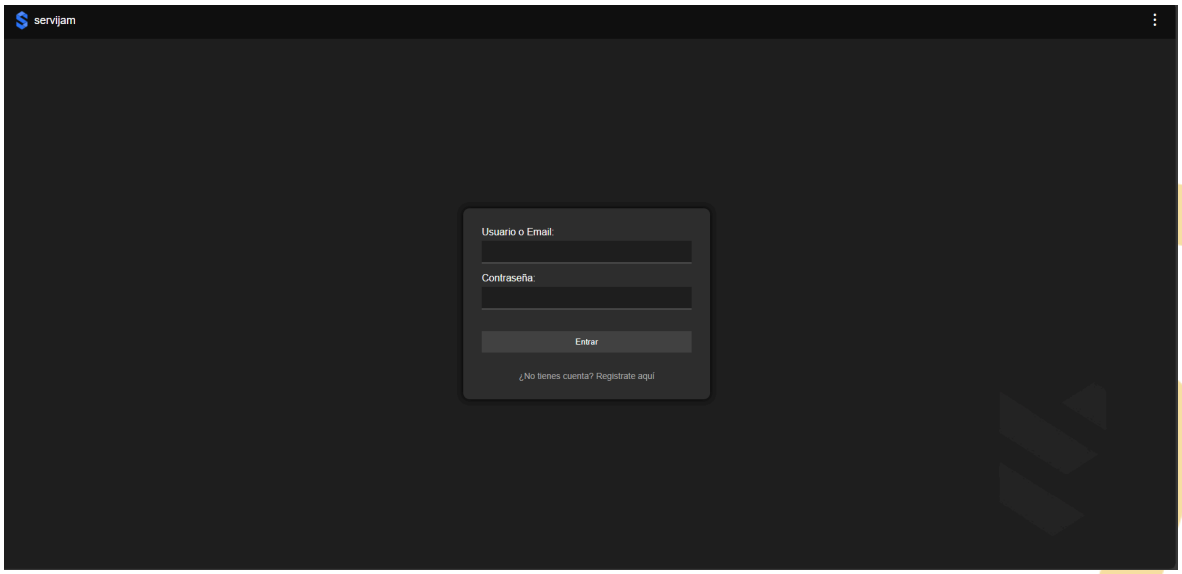


Este menú nos ofrece las siguientes opciones:

Acerca de nosotros. Nos lleva a un apartado donde contamos nuestro propósito con **SERVIJAM**.

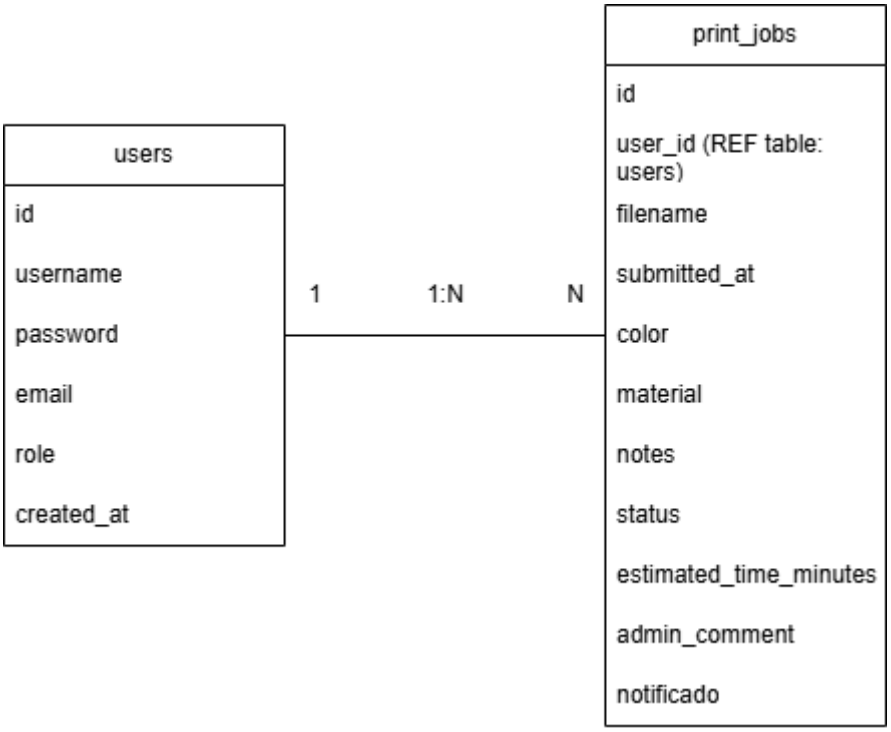


Cerrar sesión. Cierra la sesión actual **(usuario/administrador)** y redirige al inicio de sesión.



3.2. Base de datos

La base de datos de Servijam constituye el núcleo estructural para la gestión de usuarios, solicitudes de impresión y su trazabilidad. Se ha diseñado bajo un modelo relacional implementado sobre MariaDB, dada su ligereza, compatibilidad con PHP y capacidad de escalado en entornos de producción.



Modelo entidad-relación

El diseño consta de dos entidades principales: users y print_jobs, conectadas mediante una relación **uno a N (1:N)**. Esto refleja que un usuario puede enviar múltiples solicitudes de impresión, pero cada solicitud pertenece a un único usuario.

La estructura está pensada para garantizar integridad referencial, eficiencia en las búsquedas, y permitir futuras ampliaciones como historial, estadísticas o control de múltiples impresoras.

Tabla users

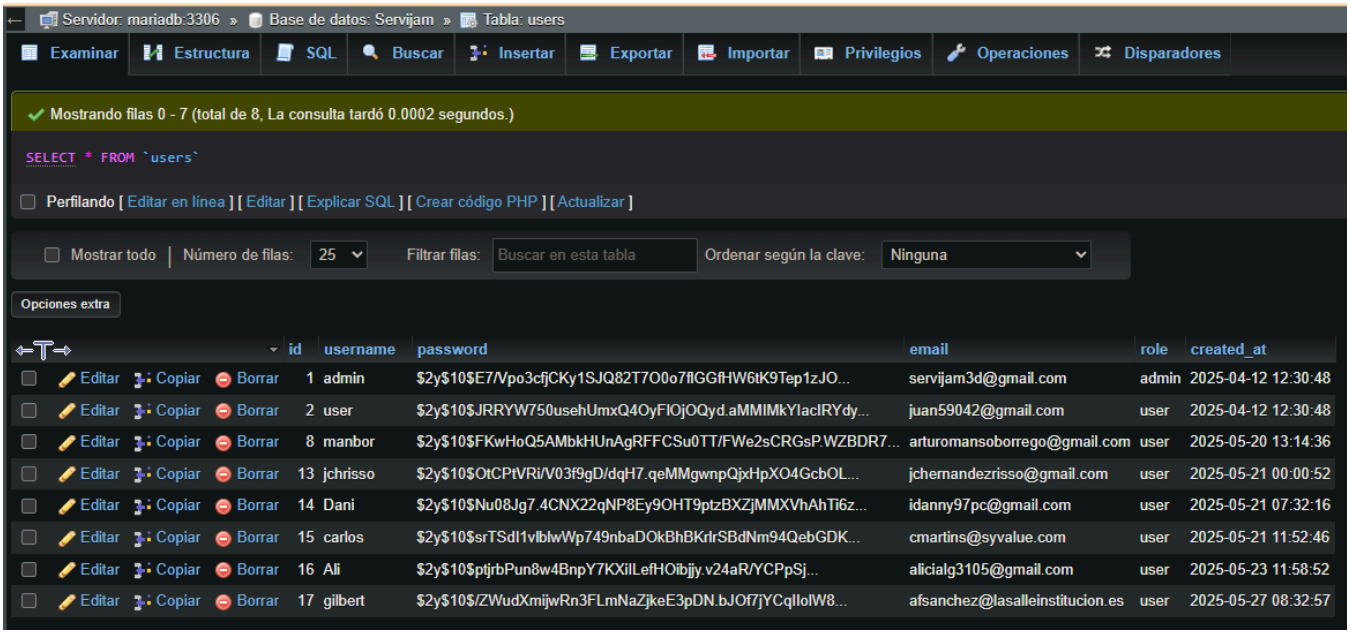


Tabla encargada de almacenar la información de los usuarios autenticados, incluyendo tanto clientes como administradores.

Campo	Tipo de dato	Descripción
id	INT (PK, AUTO_INCREMENT)	Identificador único del usuario.
username	VARCHAR(50)	Nombre de usuario (unique).
password	VARCHAR(255)	Contraseña cifrada con hash.
email	VARCHAR(100)	Correo electrónico (unique).
role	ENUM('user', 'admin')	Rol del usuario.
created_at	TIMESTAMP	Fecha de registro.

Tabla print_jobs

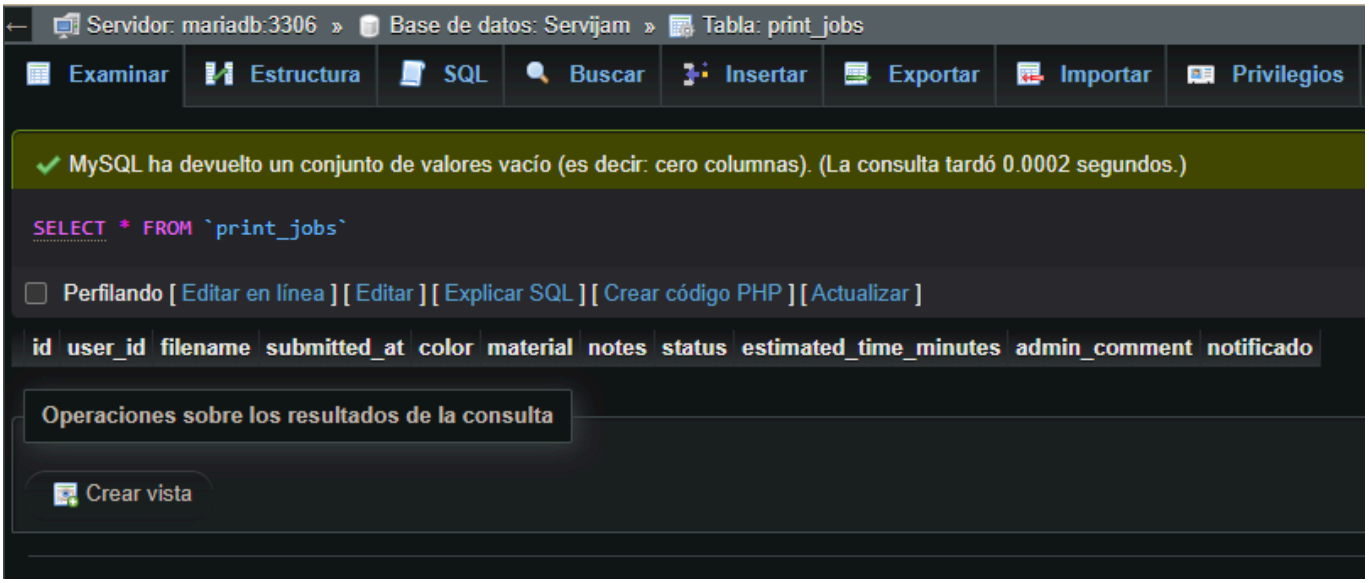


Tabla que contiene cada solicitud de impresión enviada por un usuario. Se incluye información técnica como color, material, comentarios administrativos y estado del proceso.

Campo	Tipo de dato	Descripción
id	INT (PK, AUTO_INCREMENT)	Identificador único del usuario.
user_id	INT (FK → users.id)	Usuario que realiza la solicitud.
filename	VARCHAR(255)	Nombre del archivo G-code asociado.
submitted_at	TIMESTAMP	Fecha y hora de envío de la solicitud.
color	CHAR(7)	Color solicitado en formato hexadecimal (ej. #ffffff).
material	VARCHAR(30)	Tipo de material deseado (ej. PLA, ABS, PETG).
notes	TEXT	Descripción o requerimientos adicionales del usuario.

status	ENUM('pendiente', 'en_proceso', 'completada', 'rechazada')	Estado actual del trabajo.
estimated_time_minutes	INT	Estimación del tiempo total de impresión en minutos.
admin_comment	TEXT	Comentarios del administrador (opcional).
notificado	BOOLEAN	Indica si el usuario fue notificado por correo (true/false).

Justificación del diseño

Separación de responsabilidades: cada tabla maneja su conjunto de datos específico.

Seguridad: las contraseñas se almacenan cifradas.
Escalabilidad: se puede adaptar fácilmente a múltiples impresoras, usuarios y una cola de impresión.
Extensibilidad: es posible añadir tablas adicionales como printers, logs o historial sin afectar la arquitectura base.

Capítulo 4
CONCLUSIONES Y LÍNEAS FUTURAS

4.1. CONCLUSIONES

El desarrollo del proyecto **SERVIJAM** ha permitido demostrar la viabilidad técnica y funcional de implementar un **sistema de gestión** de impresión 3D en red, **centralizado**, **seguro** y **accesible** desde múltiples dispositivos.
A lo largo de la planificación, implementación y validación del sistema, se han cumplido satisfactoriamente los objetivos propuestos, garantizando una **solución robusta** y **adaptable** a diferentes entornos como centros educativos, laboratorios de prototipado o espacios maker.

Desde una perspectiva técnica, el uso de tecnologías como **Klipper**, **Moonraker**, **Fluidd**, y la integración de herramientas como **Docker**, **Tailscale** y **DuckDNS** ha dotado al sistema de una arquitectura modular, segura y altamente configurable. Además, el uso de una **Raspberry Pi** como nodo principal ha demostrado ser una solución eficiente en términos de coste y consumo energético, sin comprometer el rendimiento del sistema.

En cuanto al desarrollo web, se ha logrado implementar una interfaz intuitiva para usuarios y administradores, permitiendo la gestión completa del flujo de impresión: desde el registro y envío de archivos hasta la monitorización y control en tiempo real. Esta interfaz ha sido diseñada teniendo en cuenta criterios de **usabilidad**, **trazabilidad de las solicitudes** y **control de acceso**, lo que garantiza una experiencia de usuario satisfactoria.

El sistema ha sido probado en un **entorno real simulado**, validando la correcta interacción entre los distintos componentes de **software** y **hardware**. Asimismo, la integración con una red ya existente se ha llevado a cabo sin generar conflictos ni comprometer la seguridad del entorno, cumpliendo con los principios de compatibilidad y escalabilidad establecidos.

En resumen, **SERVIJAM** representa una solución eficaz, replicable y escalable para la gestión de servicios de impresión 3D en red, con potencial para ser implementada en escenarios reales que demanden control, accesibilidad y eficiencia.

Finalmente, este proyecto no solo ha representado un reto técnico, sino también una valiosa oportunidad de **aprendizaje colaborativo**. La **coordinación** entre los distintos roles involucrados, el **trabajo en equipo** y la **resolución de incidencias** en tiempo real han sido fundamentales para alcanzar un **resultado profesional**, **útil** y **alineado** con las necesidades actuales del sector tecnológico.

4.2.

A lo largo del desarrollo de Servijam, se ha puesto de manifiesto el potencial de esta plataforma como base para un sistema más robusto y versátil. Por tanto, se proponen las siguientes líneas de trabajo a futuro con el fin de evolucionar y escalar el proyecto:

Control total de una granja de impresoras

Una evolución natural del sistema consiste en ampliar su capacidad para gestionar múltiples impresoras 3D de forma simultánea.

Esto implicaría:

- Desarrollar una arquitectura de control distribuido que permite gestionar varias instancias de Klipper/Moonraker desde una única interfaz centralizada.
- Implementar balanceo de carga para asignar trabajos de impresión a la impresora más adecuada o disponible.
- Monitorización en tiempo real y control individualizado por impresora, con métricas agregadas en el panel de administración.

Logística para diferentes sucursales o sedes

Con la consolidación del sistema en una sede, se plantea la posibilidad de extender su uso a múltiples ubicaciones, conectadas mediante redes privadas virtuales (VPN).

Para ello, se propone:

- Incorporar un sistema de gestión multinodo que permita visualizar y controlar impresoras en distintas sedes desde un único panel.
- Añadir una capa de organización por ubicación geográfica o centro de trabajo.
- Automatizar la sincronización de configuraciones y usuarios entre sedes, respetando las políticas de seguridad y privacidad.

Edición y personalización del firmware de la impresora

A medida que se amplíen las funcionalidades del sistema, podría ser beneficioso modificar el firmware Klipper para adaptarlo mejor a necesidades específicas.

Las líneas a seguir incluirían:

- Personalizar macros y comandos de Klipper para optimizar procesos internos.
- Añadir compatibilidad con sensores o periféricos adicionales que mejoren la fiabilidad y seguridad de las impresiones.
- Contribuir con mejoras a la comunidad de Klipper/Moonraker, generando documentación del proceso de personalización.

Integración con sistemas de gestión académica o empresarial

Para facilitar su adopción en entornos educativos o industriales, Servijam podría vincularse a plataformas existentes:

- Sistemas de autenticación mediante Active Directory o servicios de Google Workspace.
- Exportación de estadísticas e informes para justificar el uso de recursos y justificar gastos o facilitar la toma de decisiones.

Implementación de verificación en dos pasos (2FA)

Para reforzar la seguridad del sistema y proteger el acceso tanto de usuarios como de administradores, se plantea incorporar un mecanismo de verificación en dos pasos mediante aplicaciones de autenticación como Google Authenticator, Microsoft Authenticator o Authy.

Esta mejora permitiría:

- Solicitar un código temporal (TOTP) generado desde la app del usuario al iniciar sesión.
- Asociar una clave secreta única a cada cuenta durante el proceso de activación del 2FA.
- Almacenar las claves de forma cifrada en la base de datos y permitir la desactivación segura en caso de pérdida del dispositivo.
- Aumentar la resistencia del sistema frente a ataques por robo de credenciales o accesos no autorizados desde redes externas.

Bibliografía

Tailscale y WireGuard

- Tailscale Inc. (s.f.). *What is Tailscale?* Recuperado de: https://tailscale.com/kb/1018/what-is-tailscale/
- Tailscale Inc. (s.f.). *How Tailscale Works*. Recuperado de: https://tailscale.com/kb/1133/how-tailscale-works/
- WireGuard. (s.f.). *WireGuard official documentation*. Recuperado de: https://www.wireguard.com/protocol/
- Reddit. (2023). *Tailscale vs traditional VPNs*. Recuperado de: https://www.reddit.com/r/selfhosted/comments/10ppix9/using_tailscale_for_remote_access/
- Stack Overflow. (2022). *How to establish a direct WireGuard connection through NAT*. Recuperado de: https://stackoverflow.com/questions/59437154/how-to-set-up-wireguard-through-nat

Moonraker (Klipper API)

- Moonraker Docs. (s.f.). *Moonraker REST API*. Recuperado de: https://moonraker.readthedocs.io/en/latest/
- GitHub - Klipper3d/moonraker. (s.f.). *Issues y discusiones*. Recuperado de: https://github.com/Arksine/moonraker/discussions
- Reddit. (2023). *Moonraker configuration help*. Recuperado de: https://www.reddit.com/r/klippers/comments/uh7q89/help_with_moonraker_config/

Fluidd

- Fluidd. (s.f.). *Documentación oficial*. Recuperado de: https://docs.fluidd.xyz/
- GitHub - FluiddPI. (s.f.). *Repositorio y problemas comunes*. Recuperado de: https://github.com/fluidd-core/fluidd
- Reddit. (2023). *Fluidd vs Mainsail - Comparativa y elección*. Recuperado de: https://www.reddit.com/r/klippers/comments/p1n5ng/fluidd_vs_mainsail/

Apache HTTP Server

- Apache Foundation. (s.f.). *Apache HTTP Server v2.4 Documentation*. Recuperado de: https://httpd.apache.org/docs/2.4/
- DigitalOcean. (2021). *How To Set Up Apache Virtual Hosts on Ubuntu*. Recuperado de: https://www.digitalocean.com/community/tutorials/how-to-set-up-apache-virtual-hosts-on-ubuntu-20-04
- Stack Overflow. (2020). *Apache port conflict resolution*. Recuperado de: https://stackoverflow.com/questions/10349041/apache-port-conflict

Docker & Docker Compose

- Docker Inc. (s.f.). *Docker Documentation*. Recuperado de: https://docs.docker.com/
- Docker Compose. (s.f.). *Overview of Docker Compose*. Recuperado de: https://docs.docker.com/compose/
- GitHub Discussions. (s.f.). *Running Moonraker and Fluidd with Docker*. Recuperado de: https://github.com/Arksine/moonraker/issues/399
- Stack Overflow. (2021). *How to bind ports and volumes in docker-compose.yml*. Recuperado de: https://stackoverflow.com/questions/32319396/docker-compose-ports-vs-expose

Duck DNS

- DuckDNS. (s.f.). *Documentación oficial*. Recuperado de: https://www.duckdns.org/install.jsp
- GitHub - DuckDNS. (s.f.). *Actualizaciones automáticas con scripts*. Recuperado de: https://github.com/duckdns/duckdns
- Reddit. (2022). *DuckDNS setup in Docker with cron job*. Recuperado de: https://www.reddit.com/r/selfhosted/comments/m64c3l/duckdns_cron_job_docker_container/